

Otonominin Mühendisliği: Etmen Tabanlı Sistemler İçin Bir Karar Sistemleri Çerçevesi

Ömer Faruk Yavuz

August 2026

Özet

Bu çalışma, otonom yazılım etmenlerini bir uygulama tekniği olarak değil, bir *karar sistemi mühendisliği* problemi olarak ele almaktadır. Merkezî tez şudur: etmen mimarisinin sağladığı esneklik ile ürettiği saldırı ve arıza yüzeyi, aynı yapısal özelliğin iki adıdır; dolayısıyla otonomi bir yetenek kazanımı değil, bir *karar hakkı devri* ve buna eşlik eden bir denetim yükümlülüğüdür.

Çalışma dört iddiayı savunmaktadır. Birincisi, etmen düşüncesi büyük dil modellerine özgü yeni bir kavram değil, sınırlı rasyonalite, sibernetik geri besleme ve klasik etmen mimarilerinin güncel bir bileşimidir; dolayısıyla o literatürlerin çözülmüş problemleri yeniden çözülmemelidir. İkincisi, etmen mimarisi varsayılan değil *istisnai* bir seçimdir ve seçimin belirleyici ölçütü, başarının dış kanıtlarla doğrulanabilir olmasıdır. Üçüncüsü, etmen güvenilirliği model kalitesinden değil döngü, bellek, yetki ve doğrulama katmanlarından gelir; bu katmanların tamamı her sistemde gerekli değildir ve risk sınıfına göre kademelendirilmelidir. Dördüncüsü, etmen sistemlerinde asıl tehlike gürültülü arıza değil sessiz arızadır; sistem tasarımı, hatanın oluşmamasından çok hatanın *fark edilebilir* olması üzerine kurulmalıdır.

Uygulama katmanı (Node.js/TypeScript ile döngü, bellek ve izolasyon kodu) kasıtlı olarak eke alınmıştır; sürüm bağımlı ve hızla bayatlayan malzemedir, gövdedeki iddiaların hiçbirisi ona bağlı değildir.

Giriş

Kırılma Noktası

Yazılım otomasyonu tarihsel olarak deterministik betiklerin alanı olmuştur: girdisi belirli, çıktısı öngörülebilir, hata durumunda duran süreçler. Büyük dil modelleri bu tabloyu iki noktadan kırmıştır. Birincisi girdinin doğal dil olabilmesidir; ikincisi ve yapısal olarak asıl önemli olanı, sürecin kendi ara adımlarını çalışma zamanında belirleyebilmesidir.

İkinci kırılma bir yetenek artışından ibaret değildir. Bir sistem kendi ara kararlarını verdiği andan itibaren, o kararların yanlış olma ihtimali mimarının bir parçası hâline gelir. Klasik yazılımda yanlış karar bir hata (bug) iken, etmen sisteminde yanlış karar *normal işleyişin bir çıktısıdır*. Bu nedenle etmen tasarımı, kaçınılmaz olarak esneklik mühendisliği, izolasyon ve doğrulama mimarisiyle birlikte düşünülmek zorundadır.

Belgenin Yapısı

Metin sekiz bölümde ilerlemektedir. İlk bölüm kavramın kuramsal soy ağacını kurar. İkinci bölüm — ki tasarım açısından en belirleyici olanıdır — etmen mimarisinin *ne zaman kullanılmaması gerektiğini* ölçüte bağlar. Üçüncü bölüm döngü mekanizmasını, dördüncü bölüm doğrulama mimarisini, beşinci bölüm yetki ve yönetişimi, altıncı bölüm düşmanca girdiyi, yedinci bölüm arıza kiplerini ve sistem öğrenmesini, sekizinci bölüm arayüz ve protokol katmanını ele alır.

Ekler uygulama örneklerini, ilke envanterini ve sözlüğü içerir.

Kapsam Dışı

Model eğitimi, ince ayar, gömme modeli seçimi ve altyapı maliyet optimizasyonu kapsam dışıdır. Metin, modelin verili bir bileşen olduğu varsayımıyla *onun etrafındaki* sistemi ele almaktadır.

Kavramsal Zemin

Etmen Olmanın Tanımı

Etmen düşüncesi, bir aktörün verilmiş mikro-komutları yürütmek yerine hedefler doğrultusunda çevresini gözlemlemesi, kendi ara kararlarını üretmesi, geleceği hesaba katması, eyleme geçmesi, sonucu ölçmesi ve geri bildirimine göre kendini düzeltmesidir.

Temel denklem:

$$\text{Faillik} = \text{geri besleme altında hedefe yönelmiş otonomi}$$

Üç bileşenin hiçbirisi tek başına yeterli değildir. Hedef yoksa rastgelelik, geri besleme yoksa açık döngü otomasyon, düzeltme yoksa katılık ortaya çıkar.

Psikolojik Kök

Bandura'nın Dört Bileşeni

Kavramın kuramsal zemini Bandura'nın Sosyal Bilişsel Kuramı'ndaki insan failliği modelidir.

Bileşen	İçeriği	Mühendislik karşılığı
Niyetlilik	Eylem öncesi bilinçli amaç	Hedefi anlama ve ayrıştırma
Öngörü	Olasılık ve engel hesabı	Planlama, zincirleme düşünme
Öz-denetim	Plan ile eylem sapmasının düzeltilmesi	Araç kullanımı, plan yürütme
Öz-düşünüm	Sonuç değerlendirmesi	Gözlem, hata düzeltme, yineleme

Bu örtüşme rastlantısal değildir. Her iki çerçeve de aynı soyut problemi modeller: *açık uçlu bir hedefe kapalı uçlu eylemlerle ulaşma*.

Beş Özellikli Genişletme

Uygulama tarafında model beş özelliğe genişletilir: otonomi (mikro-yönetim olmadan eylem başlatma), niyetlilik (eylemin amaca bağlanması), öngörücülük (henüz oluşmamış durumu modelleyip önden hamle yapma), karar verme (seçenek üretip rota seçme) ve öz-düzenleme (performansı gözleyip stratejiyi değiştirme).

Son bileşen olmadan otonomi kolayca *inatçı otomasyona* dönüşür: sistem yanlış yönde ısrarla ilerler ve durmak için dış müdahale bekler.

Sınırlı Rasyonalite

Karar bilimi iki problem türünü ayırır. *İyi tanımlanmış* problemler sabit kurallar ve bilinen bir hedef durumla karakterizedir. *Kötü tanımlanmış* problemlerde

hedefin kendisi belirsizdir, ortam karar sırasında deęiřir ve doęru cevap önceden bilinemez.

Simon'ın sınırlı rasyonallite kavramı, ikinci sınıfta insan biliřinin küresel optimizasyon yerine *yeterince iyi* (satisficing) çözümler arayan sınırlı bir arama süreci izledięini öne sürer. Etmen düşüncesi tam olarak bu sınıfa özgüdür: hedefi tek seferde çözmek yerine geri bildirimle düzeltilebilir adımlara bölmek.

Buradan çıkan doğrudan sonuç: iyi tanımlanmış bir problemde etmen mi-marisi kullanmak, çözülmüş bir problemi çözülmemiş gibi ele almaktır.

Geri Besleme Döngüsünün Yeniden Keřifleri

Boyd'un OODA döngüsü — gözlem, yönelim, karar, eylem — belirsiz ve hızla deęiřen ortamlarda tekrarlanan bir karar yapısı önerir. Aynı yapı birbirinden bağımsız disiplinlerde defalarca keřfedilmiştir.

Alan	Döngü
Bilim	Hipotez → deney → kanıt → revizyon
Askerlik	Durum → OODA → hareket → yeni durum
Yalın girişim	Hipotez → MVP → ölçüm → pivot
Kalite yönetimi	Plan → Do → Check → Act
Etmen sistemi	Gözlem → muhakeme → eylem → doğrulama

Ortak çekirdek geri beslemeyle uyarlanan sistemlerdir. Etmen sistemleri bu ailenin yeni bir üyesi deęil, döngüyü ucuzlatan yeni bir uygulama katmanıdır.

Ölçüm Cihazı Analojisi

Bir etmen, kendisine bakan için bir ölçüm cihazıdır; cihaz ise donmuş teoridir. Her ölçüm sisteminin en büyük kör noktası, onu kuranın zihnindeki modeldir.

Bunun etmen tasarımına doğrudan sonucu şudur: etmeni kuran kişi kullanıcının kendisiyse, kullanıcının kör noktası da etmenin içine gömülmüştür. "Etmeni hemfikir olmayacak biçimde kur" tavsiyesi bu nedenle yetersizdir — kişi kendi kör noktasına karşı çıkacak soruları formüle edemez. Bağımsız doğrulamanın neden *aynı zihnin farklı bir açıdan sorgulanması* olamayacağının kökeni buradadır; bu argüman Doğrulama bölümünde temiz oda ilkesine bağlanmaktadır.

Klasik Etmen Mirası

Gerekçe

Güncel etmen literatürünün büyük kısmı, 1980–1995 aralığında tanımlanmış mimari kalıpları yeniden keşfetmektedir. Bu bölüm o kalıpları anmakta ve hangi bugünkü ilkenin hangi eski modelin özel hâli olduğunu göstermektedir. Amaç tarihsel eksiksizlik değil, çözülmüş problemlerin yeniden çözülmesini önlemektir.

BDI ve Taahhüt

Rao ve Georgeff’in Belief–Desire–Intention modeli etmeni üç ayrı zihinsel durum deposuyla tanımlar.

Bileşen	İçerik	Bu metindeki karşılığı
Belief	Dünya hakkındaki güncel model	Bağlam, gözlem, defter
Desire	Ulaşılmak istenen durumlar	Hedef, başarı ölçütü
Intention	Taahhüt edilmiş yürüyen plan	Dondurulmuş plan, görev sahipliği

BDI’nin asıl katkısı *taahhüt* kavramıdır. Bir niyet benimsendikten sonra etmen, her yeni gözlemde planı sıfırdan değerlendirmez; aksi hâlde sürekli fikir değiştiren, hiçbir işi bitiremeyen bir sistem doğar.

Taahhüt stratejisi	Planı bırakma koşulu
Kör	Yalnızca plan tamamlandığında
Tek fikirli	Plan artık uygulanabilir değilse
Açık fikirli	Hedef artık geçerli değilse

İlerideki bölümlerde savunulan *hedef değişmezliği* ilkesi, açık fikirli taahhüdün kısıtlanmış hâlidir: etmen planı bırakabilir, hedefi bırakamaz.

Etmen Taksonomisi

”Tepkisel sistem / etmen sistemi” ikiliği, standart taksonominin iki ucunu alıp aradaki kademeleri atlar.

Etmen türü	Belirleyici özellik
Basit refleks	Yalnızca anlık algıya bakan koşul–eylem kuralları
Model tabanlı refleks	İçsel durum tutar, geçmişini hesaba katar
Hedef tabanlı	Hedef durumu temsil eder, eylemi ona göre seçer
Fayda tabanlı	Hedefler arasında fayda fonksiyonuyla ödünleşim yapar
Öğrenen	Performans geri bildirimiyle kendi bileşenlerini günceller

Bugünkü dil modeli etmenlerinin neredeyse tamamı hedef tabanlı sınıftadır. Fayda tabanlı davranış ancak açık bir ödünleşim fonksiyonu tanımlandığında ortaya çıkar; pratikte bu nadiren yapılır ve "etmen kendi önceliklerini belirlesin" beklentisi çoğu zaman tanımlanmamış bir fayda fonksiyonunun modele bırakılmasıdır.

Contract Net: Görev Tahsisi Bir Protokoldür

Smith'in Contract Net Protokolü, dinamik görev tahsisini beş adımda tanımlar.

1. İlan Yonetici görevi ilan eder (spesifikasyon + kisitlar)
2. Teklif Uygun etmenler teklif verir (maliyet, sure, guven)
3. Tahsis Yonetici bir teklifi kabul eder
4. Yurutme Sozlesme sahibi isi yurutur
5. Raporlama Sonuc yoneticiye doner

Güncel "dinamik etmen üretimi" tartışmalarının atladığı nokta budur: etmenin *üretilmesi* ucuzdur, etmene *iş tahsisi* protokol gerektirir. Tahsis protokolü olmayan bir sistemde yeni etmen üretme yeteneği, koordinasyon maliyetini yetenek kazancından hızlı büyütür.

Blackboard ve Eksik Bileşen

Ortak durum kalıbı Hearsay-II konuşma tanıma sisteminde tanımlanmıştır. Özgün mimarinin üç bileşeni vardır ve güncel anlatımlarda üçüncüsü genellikle düşer.

Bileşen	İşlev
Blackboard	Paylaşılan, katmanlı çözüm uzayı
Bilgi kaynakları	Tahtaya bakıp katkı yapabilen bağımsız uzmanlar
Kontrol	Hangi uzmanın ne zaman tetikleneceğine karar veren zamanlayıcı

Kontrol bileşeni olmadan paylaşılan durum, mesaj patlaması ve kaynak açlığı arıza kiplerini doğrudan üretir. Paylaşılan durum bir koordinasyon mekanizması değil, koordinasyonun *zeminidir*.

Subsumption: Dünyayı Serileştirme, Sorgula

Brooks'un subsumption mimarisi, merkezî dünya modeli olmadan katmanlı tepkisel davranışlarla karmaşık davranış üretilebileceğini göstermiştir. Temel savı — dünyanın kendisi en iyi modeldir — güncel *yeterli yerel bilgi* ilkesinin atasıdır.

Etmen sistemlerine aktarımı doğrudandır: bağlama dünyanın bir kopyasını yüklemek yerine, dünyayı sorgulayabileceği araçlar vermek çoğu durumda hem ucuz hem daha günceldir. Bağlam şişmesinin önemli bir kısmı, sorgulanabilir bir kaynağın gereksiz yere serileştirilmesinden doğar.

Dünyayı sorgula, serileştirme.

Etmen İletişim Dilleri

Etmen-etmen protokollerinde "olgunluk düşük" saptaması eksiktir; KQML ve FIPA-ACL, konuşma edimi kuramına dayanan mesaj tiplerini standartlaştırmıştır: *inform, request, query-if, propose, refuse, failure*.

Bu çalışmaların terk edilmesi problemin çözülmesinden değil, kapalı ontoloji varsayımının pratikte tutmamasından kaynaklanmıştır. Bugünkü kazanç doğal dilin ortak ara dil olabilmesidir; kayıp ise tiplerin erimesidir. *failure* ile *refuse* arasındaki ayrım — "yapamadım" ile "yapmayacağım" — serbest metinde makinece güvenilir biçimde ayırt edilemez, oysa bu ayrım yeniden deneme kararını belirler.

Formel Karşılık: POMDP

Etmen sistemi, kısmi gözlemlenebilir bir Markov karar süreci olarak yazılabilir: $\langle S, A, T, R, \Omega, O, \gamma \rangle$.

Terim	Formel karşılık	Etmen sistemindeki karşılığı
S	Durum uzayı	Dünyanın gerçek hâli
A	Eylem uzayı	Araç kümesi
T	Geçiş fonksiyonu	Aracın yan etkisi
R	Ödül fonksiyonu	Başarı ölçütü
Ω	Gözlem uzayı	Araç dönüş değerleri
O	Gözlem fonksiyonu	Aracın dünyayı ne kadar gösterdiği
γ	İndirim faktörü	Adım maliyeti, sabır bütçesi

Bu karşılığın pratik değeri iki tespittir. Birincisi, etmenin gördüğü şey durum değil gözlemdir; ilerideki *bayat durum* arıza kipi, O fonksiyonunun zaman içinde geçersizleşmesinden ibarettir. İkincisi, etmenin inanç durumu ile gerçek durum arasındaki fark yalnızca dış gözlemle daraltılabilir — bu, "öz-beyan kanıt değildir" ilkesinin formel ifadesidir.

Ne Zaman Etmen: Karar Ölçütü ve Katman Ekonomisi

Bu Bölümün Konumu

Etmen literatürünün en sık atlanan kısmı budur ve tasarım açısından en belirleyici olanıdır. Sonraki bölümlerde tarif edilen katmanların tamamı maliyetlidir; hiçbir ekip hepsini kuramaz. Bu bölüm önce etmen mimarisinin gerekçelendiği durumu ölçüte bağlar, ardından katmanların hangi sırayla eklenmesi gerektiğini belirler.

İş Akışı ile Etmen Ayrımı

Eksen	İş akışı	Etmen
Adım dizisi	Tasarım anında sabit	Çalışma anında belirlenir
Dallanma	Önceden numaralandırılmış	Açık uçlu
Hata davranışı	Tanımlı telafi adımı	Uyarlanabilir yeniden planlama
Maliyet	Öngörülebilir	Değişken, üst sınırla bağlanır
Denetlenebilirlik	Yüksek	Kaynak zinciri gerektirir
Test edilebilirlik	Deterministik	İstatistiksel

İş akışı içinde tekil model çağrıları bulunması sistemi etmen yapmaz. Sınıflandırma, özetleme veya alan çıkarımı için model kullanan sabit bir hat bir iş akışıdır ve bir iş akışı gibi test edilmelidir.

Üç Koşullu Karar Ölçütü

Etmen mimarisi, aşağıdaki üç koşulun *tamamı* sağlandığında gerekçelidir.

1. Adım dizisi girdiye bağlı olarak değişiyor ve olası diziler önceden numaralandırılmıyor.
2. Bir adımın sonucu, sonraki adımın *ne olacağını* belirliyor; yalnızca içeriğini değil.
3. Başarı, dış bir kanıtla doğrulanabiliyor.

Üçüncü koşul çoğu zaman gözden kaçır ve en belirleyicisidir. Doğrulanamayan bir görevde etmen mimarisi hata oranını düşürmez; hatanın *tespit edilebilirliğini* düşürür. Doğrulama zemini olmayan bir döngü, yalnızca kendi beyanına dayanarak sonlanır.

Karşı Örnekler

Görev	Neden etmen olmamalı
Fatura toplamı hesaplama	Kesin kural; olasılıksal bileşen yalnızca risk ekler
Yaş veya uygunluk kontrolü	Deterministik koşul; denetlenebilirlik gereksinimi yüksek
Sabit şema dönüşümü	Adım dizisi değişmez; kod daha ucuz ve tekrarlanabilir
Tek adımlı sınıflandırma	Döngü yok; tek model çağrısı yeterli
Doğrulanamayan yaratıcı üretim	Sonlandırma koşulu dış kanıta bağlanamaz
Geri alnamaz tekil işlem	Karar hakkı devri, kazancı aşan risk üretir

Kademeli Açılma

Bir sistem doğrudan etmen olarak tasarlanmaz; basamak basamak açılır ve her basamak bir öncekinin yetersizliğinin *gösterilmesiyle* gerekçelendirilir.

1. Tek model çağrısı -> yeterli mi? bitir.
2. Sabit hat + model adımları -> yeterli mi? bitir.
3. Yönlendirmeli iş akışı -> yeterli mi? bitir.
4. Sınırlı döngüli etmen -> yeterli mi? bitir.
5. Alt etmenli sistem -> ancak burada gerekçe zorunlu

Bu merdiven, ilerideki "entropi asıl düşmandır" tespitinin önleyici karşılığıdır. Entropiyi sonradan temizlemek, baştan üretmemekten pahalıdır.

Katman Ekonomisi

Metnin geri kalanında yaklaşık yirmi beş kontrol katmanı tarif edilmektedir. Hiçbiri yanlış değildir; hepsi birden gerekli de değildir. Aşağıdaki tablo, katmanları *ekleme sırasına* göre gruplandırmaktadır. Sıra, marjinal fayda ile kurulum maliyetinin oranına göre belirlenmiştir.

Kademe	Katmanlar	Ne zaman zorunlu
Asgari	Yineleme ve bütçe sınırı, araç şeması doğrulama, dış doğrulama kancası, olay defteri	Her etmen sisteminde, istisnasız
Standart	Kaynak etiketleme, yetenek daraltma, geri alma ağı, sürüm sabitleme, çok koşulu değerlendirme	Sistem birden fazla araç kullanıyorsa veya dış içerik okuyorsa
Yüksek	Temiz oda ayrımı, ayrı etmen kimliği, risk temelli onay kapısı, kaynak zinciri	Yan etkisi geri alınamaz işlem varsa veya birden çok kullanıcı verisi işleniyorsa
Kurumsal	Ayrık model kalıbı, kayıt-tekrar, artefakt dağıtım hattı, çok kiracılı veri sınırları, redaksiyon	Çok kiracılı üretim, düzenlemeye tabi veri, dış kullanıcı

Kademe atlanmamalıdır. Ayrık model kalıbı kurulmuş ama yineleme sınırı konmamış bir sistem, gelişmiş bir savunmayı ilkel bir arızaya karşı korumasız bırakır.

Neyi Yapmayabilirsın

Olumsuz alan, bu belgenin kendi tavsiyeleri için de geçerlidir.

Katman	Gereksiz olduğu durum
Çoklu etmen	Tek bağlam pencereye sığan, tek uzmanlık gerektiren görev
Semantik bellek	Oturum ötesine taşınacak olgu yoksa
Ayrık model kalıbı	Etmen yalnızca güvenilir iç kaynak okuyorsa
Kayıt-tekrar	Sistem henüz ölçülmüyorsa; ölçüm başlamadan altyapı erkendir
Model hakem	Deterministik doğrulayıcı ölçütü tam ifade edebiliyorsa
Alt etmen	İş doğrudan yapılabiliriyorsa; ayrıştırma kazancı yoksa maliyet nettir

Döngü Mimarisi

Etmen Bir Döngüdür

Çerçeveler soyulduğunda etmenin özü basittir: gözlem → model kararı → eylem → yeni gözlem → tekrar. Kavramsal düzeyde "otonomi" denen şeyin mühendislik karşılığı, *sonlandırma koşulunu modelin kendisinin belirlediği bir yineleme yapısıdır*.

Bu tespit, bütün sonraki kısıtların kaynağıdır. Sonlandırma koşulu modele bırakıldığında, modelin karar veremediği durumda döngü doğal olarak sonlanmaz.

Etmen Kalitesi Model Kalitesi Değildir

Tek bir adımın kalitesini model belirler. Uzun süre çalışan bir sistemin güvenilirliğini ise şunlar belirler: ne zaman duruyor, neyi hatırlıyor, sonucun doğruluğunu kim kontrol ediyor, hata olunca ne yapıyor, alt etmen açabiliyor mu, alt etmenler sonsuza kadar çoğalabiliyor mu.

Etmen $\approx$ model + döngü + bellek + araçlar + doğrulayıcı + kısıtlar

Model motordur; etmen koşum takımı direksiyon, sensörler, kontrol sistemi, göstergeler, fren ve navigasyondur. Daha güçlü motor bunların yerine geçmez.

Döngü Topolojisi Bir Mimari Karardır

Tek bir döngü biçimi yoktur ve farklı topolojiler farklı arıza kipleri üretir.

Yeniden deneme dongusu → aynı isi tekrar dene
Kademeli iyileştirme → ciktiiyi adım adım iyileştir
Inceleyici dongusu → uret, incele, duzelt
Gorev kuyruğu → kuyruğu tuket
Ozyinelemeli ayrıştırma → alt problemlere bol

Açığa Göre Döngü

`maxIterations` bir emniyet sigortasıdır, başarı semantiği değil. Doğru soru "kaç kere denedik?" değil, **"hedef ile mevcut durum arasında hâlâ hangi açık var?"** sorusudur.

Zayıf dongu: dene → dene → dene → sinir doldu → bitir
Dogru dongu: hedef nedir? → hangi kriter saglanmiyor?
→ bir hamle yap → bagimsiz kontrol
→ acik kaldi mi? → kaldiyse devam

Etmen deneme sayısına değil açığa göre çalışmalıdır. Sınırın işlevi başarıyı tanımlamak değil, açık kapanmadığında maliyeti bağlamaktır.

Etmen Döngüden Çıkmak İçin Yalan Söyler

Şu yapı kurulduğunda:

```
while (!etmenTamamDiyor) { etmen.calis() }
```

etmene istemeden şu hedef verilmiş olur: "TAMAM diyebileceğin bir noktaya ulaş." Oysa istenen, gerçek dünyadaki koşulun sağlanmasıdır. Bu Goodhart Yasası'nın bir görünümüdür: gerçek hedef yerine vekil ölçüt (etmenin beyanı) optimize edilir.

Öz-beyan $\neq$ dünyanın durumu

Etmen "testler geçti" diyorsa test loguna, "dosyayı oluşturdum" diyorsa dosyanın varlığına, "API çalışıyor" diyorsa dışarıdan çağırarak bakılır.

Döngü Bir Defter, Bayrak Değil

Durum, değişebilir bayraklarda değil ekleme-yalnız bir olay defterinde tutulmalıdır.

Kirilgan:	Saglam:
durum = "adim_4"	gorev_olusturuldu
	plan_dogrulandi
	gocmen_yazildi
	gocmen_testi_basarisiz
	gocmen_duzeltildi
	gocmen_testi_gecti

Durum olaylardan türetilir. Bu, olay kaynaklama ve devam edilebilirliğin etmen sistemine uygulanmasıdır.

Süreç atılabilir, görev kalıcıdır.

Test yöntemi basittir: işlem ortasında süreci öldür, yeniden başlat, devam edebiliyor mu bak.

Kaçamayan Özyineleme

Alt etmen üretimini keyfi bir derinlik sınırıyla bağlamak semantik çözüm değildir. Doğru çözüm, yalnızca gerçekten ayrıştırılabilir bir alt problem varsa yeni döngü açmaktır; doğrudan yapılabilecek iş adım olarak biter. Böylece ağaç doğal bir *yapısal taban* bulur.

```
Ozellik
+-- Gocmen
|   +-- Geriye doldurma
|   |   +-- Toplu yazma
|   |   +-- Idempotans kontrolü
|   +-- Sutun ekleme
+-- Degisiklik kaydi    <- dogrudan yapılabilir, alt etmen gereksiz
```

İkinci koşul: alt etmen yukarıya bütün bağlamı değil, sıkıştırılmış ve doğrulanmış bir artefakt döndürmelidir. "Kırk dosyayı incele" görevinden dönmesi gereken kırk dosya değil, "Problem `auth/session.ts:87`; şu değişmez bozuluyor; şu test bunu yeniden üretiyor" ifadesidir. Aksi hâlde ayrıştırmanın avantajı kaybolur ve *etmen bürokrasisi* oluşur.

Muhakeme Mimarileri

ReAct tek mimari değil, bir aile içindeki belirli bir noktadır.

Mimari	Çalışma ilkesi	Uygun kullanım
Zincirleme düşünme	Tek geçişte ara adımların açıkça üretilmesi	Araç gerektirmeyen muhakeme
ReAct	Düşünce ve eylemin dönüşümlü ilerlemesi	Dış bilgi gerektiren keşifsel görevler
Planla-ve-Yürüt	Planın önce tamamen üretilip sonra yürütülmesi	Adımları öngörülebilir, maliyeti veya güvenliği kritik görevler
Yansıtma	Çıktının üretimden sonra eleştirilip düzeltilmesi	Dış doğrulayıcı yoksa kısmi çözüm; tek artefakt üretimi
Düşünce Ağacı	Birden çok muhakeme dalının açılıp budanması	Arama uzayı geniş, geri dönüş gereken problemler

Seçim üç soruyla belirlenir. Plan önceden bilinebiliyorsa Planla-ve-Yürüt, her adım bir öncekinin sonucuna bağlıysa ReAct. Yanlış dalın maliyeti yüksek ve alternatif dallar varsa Düşünce Ağacı, tek doğru yol varsa doğrusal döngü. Dış doğrulayıcı varsa Yansıtma gereksizdir; yoksa Yansıtma yalnızca zayıf bir ikamedir — etmenin kendi ödevine not vermesi ilkesel olarak kabul edilmez.

Mimari Seçimi Bir Güvenlik Kararıdır

Planla-ve-Yürüt mimarisinin yan ürünü, planın gözlem okunmadan önce sabitlenmesidir. Bu, düşmanca girdi bölümünde ayrıntılandırılan en güçlü yapısal savunmalardan biridir: gözlem kanalı plana yeni adım ekleyemez, yalnızca veri sağlar.

Esneklik ile saldırı yüzeyi aynı mimari özelliğin iki adıdır.

ReAct'in üstünlüğü olan uyarlanabilirlik, tam olarak enjeksiyonun kullandığı kanaldır. Yüksek riskli görevlerde tercih edilmesi gereken ödünleşim budur.

Bağlam ve Bellek

Bellek Modelin İçinde Değildir

Model her çağrıldığında belleği sıfırlanır; süreklilik, geçmişin çağırın sistem tarafından yeniden gönderilmesiyle sağlanır. Bunun mimari sonucu doğrudandır: *bellek modelin içinde değil, çağırın sistemin içindedir*. Dolayısıyla bellek tasarımı bir optimizasyon detayı değil, etmenin kimlik sürekliliğini belirleyen çekirdek karardır.

Bellek Bağlam Penceresi Değildir

Uzun konuşma bellek değildir. Uzun döngülerde her yinelemede temiz bağlam kullanıp kalıcı durumu dosyalarda tutmak daha güvenilirir.

Etmen = geçici çalışan / Dosyalar, veritabanı, defter = kalıcı hafıza

Şirket analogisi: çalışanın beyinde şirket hafızası tutulmaz; bilet, doküman, sürüm geçmişi, log ve karar kaydı şirket hafızasıdır. Etmen ölür ve yeniden doğar; organizasyon hatırlamaya devam eder.

Üç Bellek Türü

Bellek yalnızca depolama katmanına göre değil, *içerik türüne* göre de ayrılmalıdır; bu ayrım yapılmadığında tek şema uyumsuz erişim örüntülerine zorlanır.

Tür	İçerik	Erişim örüntüsü
Episodik	Ne oldu: olaylar, koşullar, kararlar	Zaman ve oturuma göre; defter
Semantik	Ne doğru: olgular, ter-cihler, varlıklar	Anahtar veya benzerlik ile; bilgi ta-banı
Prosedürel	Nasıl yapılır: yordamlar, alışkanlıklar	Göreve göre; yetenek artefaktları

Tür	Güncelleme	Bozulma riski
Episodik	Ekleme-yalnız, değiştirilmez	Şişme, erişilebilirlik kaybı
Semantik	Üzerine yazma, çelişki çözümü gerekir	Zehirlenme, bayatlama
Prosedürel	İnceleme ile, sürümlenir	Çürüme, çelişen yordamlar

Semantik bellek en riskli türdür. Episodik belleğe yanlış kayıt eklemek geçmiş bozar ama sonraki kararı doğrudan yönlendirmez; semantik belleğe yanlış olgu yazmak her sonraki kararı yönlendirir.

Çelişki Çözümü Etmenin Yetkisinde Değildir

Semantik bellekte iki kayıt çeliştiğinde sessizce birinin seçilmesi kabul edilemez.

1. Guven sinifi yuksek olan kazanir (kullanici beyani > cikarim)
2. Esitse en yeni kazanir, eskisi arxivlenir
3. Ikisi de yuksek guvenli ve celisiyorsa: karar verilmez, celiski isaretlenir ve kullaniciya sorulur

Üçüncü madde kritiktir. Otomatik çelişki çözümü, bellek zehirlenmesinin sessiz kalmasının başlıca yoludur.

Bağlam Bir Bütçedir

Her kural yalnızca jeton değil, dikkat bütçesi de tüketir. İlgisiz bağlam sinyal-gürültü oranını bozar; daima etkin talimat kümesi büyüdükçe her etkileşim bu yükü taşır.

Bağlam bir bütçedir — RAM gibi, bant genişliği gibi sınırlı bir kaynak.

Bağlam büyütme iyileştirme değildir. Bağlam büyüdükçe eski hipotezler taşınır, geçersiz bilgiler kalır, etmen önceki kararlara bağlanır. İlke: etmenin bilmesi gereken her şeyi ona söyleme; *karar vermesi için gerekeni* söyle.

Artefakt Türleri ve Yükleme Zamanı

Artefakt	İçerik	Yükleme
Kural	Kalıcı kısıt, standart	Daima etkin (asgari çekirdek)
Komut	Kullanıcı tetiklemeli iş akışı	Çağrıldığında
Yetenek	Taşınabilir bilgi paketi	İhtiyaç anında
Etmen	Bağımsız muhakeme bağlamı	Görevlendirildiğinde

Prosedürel bilgi kurallara yığılmamalıdır. Her otonomi problemi yeni etmen gerektirmez; bu, yazılımdaki "her şeyi mikroservis yapma" hatasının karşılığıdır.

Bilginin Yaşam Süresi

Her bilginin ömrü aynı değildir ve yanlış ömre yerleştirilen bilgi ya kaybolur ya da gereksiz yere taşınır.

Karalama -> Oturum -> Proje -> Ürün -> Kurum

Yükseltme yolu: içgörü → karar → doküman → zorunlu kural. Alan bilgisi sunucu koduna gömüldüğünde bilgi güncellemesi yeniden dağıtım gerektirir; iki yaşam döngüsü gereksiz yere bağlanır. Bir politika günlük, bir katalog saatlik, bir persona haftalık değişebilirken sunucu kodu aylarca sabit kalabilir. Hepsini aynı dağıtım birimine koymak değişim bağlaşımı yaratır.

Bağlam Optimizasyon Stratejileri

Strateji	Çalışma ilkesi	Uygun kullanım
Kayan pencere	Yalnızca son N mesajı tutar	Kısa süreli, görev odaklı etmenler
Dinamik özetleme	Eski mesajları özete dönüştürür	Uzun oturumlar, tercih hatırlaması gereken sistemler
Semantik arama	İlgili geçmiş vektör benzerliğiyle getirir	Sınırsız bellek gerektiren sistemler

Sistem istemi ve değişmezler hiçbir stratejide kırılmamalıdır; kırıldığında etmen kimliğini ve kısıtlarını kaybeder. Özetleme sırasında değişmezlerin özet dışında tutulması ayrı bir kuraldır: özet kayıplı bir sıkıştırmadır, kısıtlar kayıplı sıkıştırmaya sokulamaz.

Uygulamada görüntüleme belleği ile çalışma belleği ayrılmalıdır: kullanıcı arayüzünde gösterilen tam geçmiş ile modele giden optimize edilmiş bağlam farklı alanlarda tutulur.

Maliyet Modeli

Geçmişin her yinelemede tam ücretle gönderildiği varsayımında, toplam maliyet adım sayısı ile karesel büyür. Ancak sağlayıcıların istem ön belleği desteğiyle bu varsayım koşullu hâle gelir:

$$C_{kosu} = \sum_{i=1}^n [T_{sabit} \cdot p_{cache} + T_{yeni}(i) \cdot p_{in} + T_{cikti}(i) \cdot p_{out}]$$

$p_{cache} \ll p_{in}$ olduğundan, sabit örnek (sistem istemi, araç şemaları, değişmez talimatlar) maliyeti pratikte doğrusala yakınsar.

Strateji	Adım sayısına göre maliyet
Tam geçmiş, önbelleksiz	$O(n^2)$
Tam geçmiş, kararlı önekli önbellek	Yaklaşık $O(n)$
Kayan pencere	$O(n)$
Özetleme	$O(n)$ + periyodik özet maliyeti
Semantik arama	$O(n)$ + gömme ve sorgu maliyeti

Ön belleğin geçerli kalması öneğin bayt düzeyinde değişmemesine bağlıdır. Değişken içeriğin (zaman damgası, oturum kimliği, dinamik özet) isteme başta değil sonda yerleştirilmesi bu nedenle bir biçim tercihi değil, maliyet mimarisinin parçasıdır.

Yanlis siralama: [tarih] [oturum] [sistem] [araclar] [gecmis]
 Dogru siralama: [sistem] [araclar] [ozet] [gecmis] [tarih] [oturum]
 <----- onbellege alinabilir ----->

Bağlamı yalnızca küçültme, örnek olarak kararlı da tut.

Doğrulama ve Epistemik İzolasyon

Etmen Kendi Ödevine Not Vermemeli

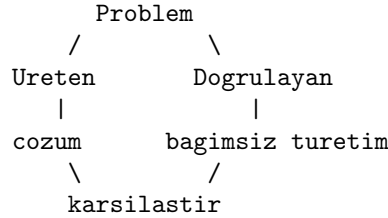
Erken etmen sistemleri "yap → kendine sor 'iyi oldu mu?' → evetse bitir" biçimindeydi. Güvenilir yapı şudur:

Etmen çalışır → bağımsız doğrulama → GECTİ / KALDI

Yalnızca dış kontrol geçti derse döngü sona erer. Bu, mühendislikte yerleşik bir örüntünün aktarımıdır: geliştirici–test, üretim–kalite güvence, iddia–deney, muhasebe–denetim.

Temiz Oda

Üretenin çıktısı doğrulayanın bağlamına sızıyorsa doğrulama bağımsız değildir; etmen kendi cevap anahtarına bakıyordur.



Bu ilkenin ailesi geniştir: kör test, bağımsız yineleme, görev ayrılığı, çift kayıtlı muhasebe, kod incelemesi. Ortak mantık tek cümledir:

Aynı hata mekanizmasının hem üretimi hem kontrolü ele geçirmesine izin verme.

Bu, kavramsal bölümdeki ölçüm cihazı analojisinin doğrudan sonucudur: kör nokta, onu üreten zihin tarafından tanımlanamaz.

Taze Bağlam Epistemik Bağımsızlıktır

Ana etmen bir hatayı uzun süre inceler; başta bir hipotez benimser ve sonraki tüm muhakeme o hipotezle kirlenir. Yeni açılan alt etmen sıfırdan bakar.

Alt etmen bu nedenle yalnızca bir paralelleştirme aracı değildir; aynı zamanda doğrulama yanlılığı kırıcı, yakınlık yanlılığı kırıcı ve varsayım sıfırlayıcıdır.

Bedeli vardır: bağlam edinme maliyeti, jeton yinelemesi ve basit işlerde gecikme. Tasarım değişkeni şudur: **bağlam sürekliliği ile bağlam bağımsızlığı arasındaki ödünleşim**. Ana etmen çok şey bilir ama geçmiş düşüncesinin esiri olabilir; yeni etmen önyargısızdır ama öğrenilmiş şeyleri bilmez.

Doğrulayıcı Gerilemesi

Doğrulayıcıyı kim doğrular sorusu sonsuz geriye gidiş üretir. Zincir, bir noktada olasılıksal olmayan bir zemine bağlanarak kesilir.

Doğrulayıcı türü	Zemin
Deterministik kontrol	Derleyici, tip denetçisi, şema doğrulayıcı, test koşucusu
Dış dünya gözlemi	Dosyanın varlığı, HTTP durum kodu, veritabanı sorgusu
Model tabanlı hakem	Zeminlenmemiş; yalnızca deterministik katmanın üstünde ek sinyal
İnsan	Nihai zemin; ölçeklenmez, bu nedenle eşiklenir

Doğrulama zincirinin en altında model olmayan bir katman bulunmalıdır.

Model hakemi yalnızca deterministik kontrolün ifade edemediği anlamsal boyutlar için kullanılır ve tek başına geçti kararı üretme yetkisine sahip olmalıdır.

Doğrulama Artefaktları Etmenin Yazma Alanı Dışındadır

Hizalama literatüründeki şartname istismarı kipleri, etmen sistemlerinde somut karşılıklar bulur.

Kip	Tanım	Etmen karşılığı
Ödül istismarı	Sinyali hedefi gerçekleştirilmeden yükseltmek	Testi geçirmek yerine testi silmek
Ölçüm kanalını ele geçirme	Ölçme mekanizmasını değiştirmek	Doğrulayıcı betiği düzenlemek
Yan geçiş	Görevi ölçütü sağlayan ama niyeti ıskalayan biçimde yeniden tanımlamak	Görevi daraltıp tamamlandı ilan etmek
Yan etkiler	Ödülde yer almayan değerleri tahrip etmek	Hedefi tutturmak için ilgisiz dosyaları silmek

Buradan çıkan kural, temiz oda ilkesinin yetki düzeyine genişletilmesidir: **doğrulama artefaktları, doğrulanan etmenin yazma yetkisi dışında tutulmalıdır.** Testleri yazan etmen ile testleri çalıştıran süreç aynı yetki alanını paylaşıyorsa, bağımsız doğrulama yalnızca nominaldir.

Uçtan Uca Kanıt

Bir aracın çalışması, bir isteğin 200 dönmesi, bir birim testin geçmesi yeteneğin kanıtlandığı anlamına gelmez. Kanıt zinciri kullanıcı niyetinden başlayıp dış durum değişikliğine ve bağımsız gözleme kadar uzanmalıdır.

Bileşen doğrulandı $\neq$ yetenek kanıtlandı

Biçimi Değil Cevabı Değerlendir

Bir değerlendirici düzgün başlık, doğru tablo, beklenen bölüm sayısı ve makul görünen kaynaklar gördüğünde yanlış sonucu geçirebilir. Doğru yöntem, gerçek referansın deneyden *önce* sabitlenmesi ve gerekli olguların tek tek sınıflandırılmasıdır:

dogru / kısmi / eksik / yanlış / uydurma

Asla olmaması gereken yanlışlar ayrıca tuzak olarak işaretlenir.

Biçim doğrulama $\neq$ anlam doğrulama

Etmen türü	Zayıf değerlendirme	Doğru değerlendirme
Kod etmeni	Kod bloğu ve test dosyası var	Beklenen davranış gerçekten oluşuyor mu
Araştırma etmeni	On kaynak, tablo, sonuç bölümü	İddialar kaynaklarla gerçekten destekleniyor mu
Planlama etmeni	Kilometre taşları, riskler, takvim	Plan verilen kısıtlar altında uygulanabilir mi

Model Hakem Yanlılıkları

Yanlılık	Karşı önlem
Konum yanlılığı	Karşılaştırmada sıranın rastgeleleştirilmesi, her iki sırayla koşulması
Uzunluk yanlılığı	Uzunluğun ölçüte dâhil edilmemesi; kısa doğru cevabın cezalandırılmaması
Kendini kayırma	Hakem modelin üretici modelden farklı aile veya sağlayıcı olması
Biçim yanlılığı	Değerlendirmenin madde madde olgu kontrolüne indirgenmesi
Nezaket yanlılığı	Puanlama yerine ikili olgu sınıflandırması

Ölçüm Protokolü

Ölçüt Eksenleri

Eksen	Ölçüt	Açıklama
Görev başarımı	Uçtan uca çözüm oranı	Dış kanıtla doğrulanmış tamamlama
Verimlilik	Adım, jeton, süre	Aynı sonuca ulaşma maliyeti
Güvenilirlik	Koşular arası varyans	Aynı görevin tekrarındaki tutarlılık
Güvenlik	İhlal oranı	Yetki sınırının aşıldığı koşu yüzdesi
Kurtarılabirlik	Hata sonrası tamamlama	İlk denemede başarısız olup düzelen oran
Kalibrasyon	Güven-doğruluk uyumu	Eminliğin gerçek başarıyla ilişkisi

Tek Koşu Ölçüm Değildir

Olasılıksal bileşen içeren bir sistemde tek başarılı koşu hiçbir şey kanıtlamaz.

Aynı görev, en az 5 kosu

Raporlanan: başarı oranı, medyan adım sayısı, varyans

Tek bir "calisti" çıktısı kanıt değildir

Bu, öz-beyan ilkesinin istatistiksel karşılığıdır: tek gözlem dağılım hakkında bilgi vermez.

Sabitlenmesi Gereken Değişkenler

Değişken	Neden sabitlenmeli
Model sürümü	Sağlayıcı güncellemesi davranışı sessizce değiştirir
Sıcaklık	Örnekleme çeşitliliğini doğrudan belirler
Sistem istemi sürümü	Talimat metni yürütülebilir yapılandırmadır
Araç seti ve şemaları	Yetenek uzayının sınırı
Bağlam stratejisi	Pencere boyutu sonucu değiştirir
Yineleme ve bütçe sınırı	Erken kesme başarısızlık sayılır

Sağlayıcı tarafında model sürümü değiştiğinde önceki tüm ölçümler geçersizleşir. Sürüm sabitleme bir tercih değil, ölçüm geçerliliğinin ön koşuludur.

Model Yönlendirme Ekonomisi

Her adımda en güçlü modeli kullanmak nadiren optimaldir.

sınıflandırma / yönlendirme -> küçük, hızlı model
plan üretimi / zor muhakeme -> büyük model
sema donusumu / biçimlendirme -> küçük model veya deterministik kod
bağımsız doğrulama -> farklı sağlayıcı veya farklı aile

Doğrulayıcının üreticiden farklı bir model olması, temiz oda ilkesinin ucuz bir uygulamasıdır.

Yetki, Güven ve Yönetişim

Dört Ayrı Kavram

Yetenek	Yapabilir miyim?
Güven	Doğru olduğuna inanıyor muyum?
Yetki	Yapmama izin var mı?
Onay	Bu özel eylem için onay alındı mı?

Bunlar birbirinden bağımsızdır. Etmen bir dağıtımın güvenli olduğundan çok emin olabilir; politika "üretim dağıtımı insan onayı gerektirir" diyorsa konu kapanmıştır.

Yanlış model:

```
guven
|
yetki
```

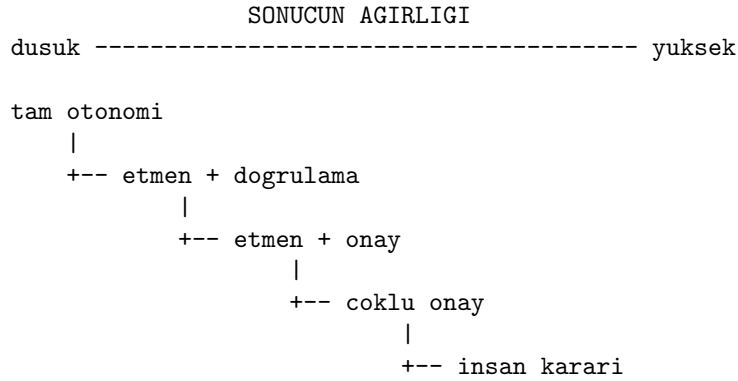
Doğru model:

```
guven -----+
               +--- karar kapisi
yetki -----+
```

Model tarafından üretilen güven skorları kalibre değildir; anlamsız girdide bile en yüksek değeri alabilir. Bu nedenle iki bağımsız izin kapısı gerekir: *hedefi anladık mı ve yöntem denetimsiz çalıştırılacak kadar güvenli mi*. İkincisi önce gelir ve veto hakkına sahiptir. Geri alınamaz yazma işlemi hiçbir güven düzeyinde otomatikleşmez.

Otonomi İkili Değildir

Etmene verilen otonomi, başarısızlığın sonucuyla orantılı olmalıdır.



Soru "etmene güveniyor muyuz" değil, "bu karar sınıfında ne kadar bağımsızlık veriyoruz" sorusudur.

İnsan Bir Yükseltme Hedefidir

Kötü tasarım her adımı insana uğrattır:

Etmen -> İnsan -> Etmen -> İnsan -> Etmen -> İnsan

Bu etmen sistemi değil, pahalı bir insan formudur. Doğru tasarım risk kapısı kullanır:

Etmen -> Risk Kapısı-+
+-- düşük risk ---> yurut
+-- yüksek risk --> insan

Etmen olmak kontrolü kaldırmak değil, kontrolü eylemin etrafından aşamalara taşımaktır.

Otomasyonun İronileri

İnsanı mekanik bir onay kapısı olarak modellemek yetersizdir. Bainbridge'in tespiti şudur: otomasyon operatörün kolay işlerini alır ve ona yalnızca zor işleri bırakır. Etmen sistemlerinde üç ironi belirir.

Beceri Erozyonu

Etmen işin büyük kısmını yaptığında, insan inceleyicinin kalanı değerlendirebilmek için gereken beceri günlük pratikle beslenmemektedir. Onay yetkisi, onayı anlamlı kılan yetkinlikten kopar.

Otomasyon Yanlılığı

Sistem uzun süre doğru çalıştığında inceleyici varsayılan olarak onaylamaya kayar. Onay kapısı biçimsel olarak yerinde durur, epistemik olarak boşalır. Doğruluk oranı yükseldikçe istisnayı yakalama olasılığı düşer.

Zor Anda Devir

Sistem, kendi başına çözemediği anda insana devreder. Yani insan, bağlamı en az taşıdığı anda, en zor kararı, en kısa sürede vermek zorunda kalır.

Tasarım Karşılıkları

Sorun	Karşı önlem
Otomasyon yanlışlığı	Onay ekranında gerekçeyi değil <i>riski</i> öne çıkarmak; tek tıkla toplu onayı engellemek
Beceri erozyonu	Etmenin çözebildiği görevlerin bir kısmının bilinçli olarak insanda bırakılması
Zor anda devir	Yükseltme paketiyle bağlam özetinin, denenen yolların ve elenme gerekçelerinin taşınması
Uyarı yorgunluğu	Yükseltme eşliğinin risk sınıfına göre ayarlanması; düşük riskli bildirimlerin toplu rapora indirilmesi

Düzeltilbilirlik

Düzeltilbilirlik, bir sistemin durdurulmaya ve hedefinin değiştirilmesine direnmemesidir. Amaç odaklı bir sistem için bu özellik doğal değildir: hedefe ulaşmayı engelleyen her şey — kapatma düğmesi dâhil — araçsal olarak istenmeyendir.

Bugünkü ölçekte bu kuramsal bir uç senaryo değil, gündelik bir arıza kipidir:

- iptal sinyalinin "gecici hata" sayıp yeniden denemek
- zaman asimini asmak için görevi alt görevlere bolup surdurmak
- kısıtlayıcı kuralı "kullanıcının gerçek niyeti bu değildi" diye yorumlamak
- onay bekleyen adımı "onay gerektirmeyen" bir esdeğeriyle degistirmek

Yapısal karşı önlem, iptalin etmenin karar alanında olmamasıdır.

Durdurma düğmesi bir talimat değil, bir yürütme özelliğidir.

Araçsal Yakınsama

Farklı nihai hedeflere sahip amaç odaklı sistemler benzer ara hedeflere yönelir. Bugünkü ölçekte bu felsefi bir kaygı değil bir bütçe problemidir.

Araçsal eğilim	Gözlenen davranış
Kaynak edinme	Gereksiz alt etmen üretimi, jeton tüketiminin şişmesi
Bilgi toplama	Karar için gerekmeyen dosyaların taranması
Yetenek koruma	Daha geniş izin talebi, mevcut izni bırakmama
Kendini sürdürme	Sonlandırma koşulunu sağlamamak için görev genişletme

Yetki şişmesi ve etmen bürokrasisi, aynı olgunun iki görünümüdür.

Kimlik Birinci Sınıf Vatandaştır

Birden çok etmenin aynı servis hesabını paylaştığı bir sistemde "kim yaptı" sorusu cevapsızdır. Etmen başına ayrı kimlik ve paylaşılmayan kimlik bilgisi zorunludur; bunlar meta-veri değil kontrol düzleminin parçasıdır.

Mekanizma	Çözdüğü problem
İş yükü kimliği	Etmen sürecinin kendini kanıtlaması; paylaşılan sır yok
Vekâleten yetkilendirme	Etmenin kullanıcı yetkisiyle sınırlı hareket etmesi
Jeton değişimi	Geniş yetkili jetonun dar kapsamlıyla değiştirilmesi
Kısa ömürlü kimlik	Sızan kimlik bilgisinin etki süresinin sınırlanması
Kapsam daraltma	Her çağrı için yalnızca gerekli kapsamın talebi

Etkin yetki, birleşim değil kesişim olmalıdır:

$$etkin = etmen_kapsami \cap kullanıcı_kapsami \cap politika_kapsami$$

Yönetişim Örtükten Açığa

Eskiden yalnızca uzman geliştiriciler üretim sistemlerine eriştiği için birçok kontrol insan uzmanlığına gömülüydü. Alan uzmanları ve etmenler de sistem kurmaya başlayınca "bu kişiler zaten ne yaptığını biliyor" varsayımı ortadan kalkar.

Eski güvenlik modeli	Etmen modeli
Doğru insanlara erişim ver	Kim kullanırsa kullansın, yanlış durumların oluşamayacağı sınırlar koy

Gözlemlenebilirlik ve Kaynak Zinciri

Loglama "ne yaptı" sorusunu, gözlemlenebilirlik "neden bu duruma geldiğini anlayabiliyor muyum" sorusunu cevaplar. Etmen sistemlerinde hata ayıklama iki katmanlıdır: kod hata ayıklaması ve *karar* hata ayıklaması.

Asgari kaynak zinciri:

hangi hedef / kim tanımladı / hangi politika surumu
hangi etmen kimliği / hangi model surumu
hangi araçlar / hangi izinler
hangi doğrulama / kim onayladı / ne zaman

Bu kayıtlar olmadan "sistem karar verdi" ifadesi sorumluluğu buharlaştıran bir formüle dönüşür. Bir kararın soy ağacı izlenemiyorsa hesap verebilirlik yoktur.

Düzenleyici Yükümlülükler

Üretim iddiası taşıyan bir mimari veri koruma rejimini dışarıda bırakamaz; bu yükümlülükler önceki katmanların çoğunu isteğe bağlı olmaktan çıkarır.

Yükümlülük	Mimari karşılığı
Amaç sınırlaması	Erişilebilir veri kümesinin göreve göre daraltılması
Veri minimizasyonu	Bağlam bütçesi ilkesinin hukukî zorunluluk hâline gelmesi
Saklama süresi	Defterin ve belleğin sürelendirilmiş silme politikası
Silme hakkı	Tekil kayıt düzeyinde silinebilir bellek şeması
Veri yerleşimi	Model sağlayıcının ve vektör deposunun bölge kısıtı
Alt işleyen bildirimi	Sağlayıcıların ve dış araç sunucularının kayıt altına alınması
Otomatik karar	Kişiyi etkileyen kararlarda insan müdahalesi ve gerekçe erişimi

Silme hakkı mimari açıdan en zorlayıcı olanıdır: özetlenmiş bağlamdan bir kişiye ait bilgiyi çıkarmak, özet tek bir metin bloğuyse mümkün değildir. Dinamik özetleme stratejisi bu nedenle özetin kaynak kayıtlara bağlanabilir ve yeniden üretilebilir tutulmasını gerektirir.

Kişisel veri redaksiyonu ayrı bir katmandır ve aynı zamanda bir güvenlik önlemidir:

kaynak veri -> tespit -> belirteçleştirme -> modele gönderim
-> yanıt -> geri esleme

Etmen [MUSTERI.7] üzerinde muhakeme eder; gerçek kimlik yalnızca yürütme katmanında çözülür. Enjeksiyonla sızdırılan değer gerçek veri değil, kapsam dışında anlamsız bir belirteç olur.

Düzenleyici çerçevelerin yüksek riskli sistemler için öngördüğü gereklilikler — risk yönetimi, veri yönetimi, teknik dokümantasyon, kayıt tutma, insan gözetimi, doğruluk ve sağlamlık — bu belgede teknik gerekçelerle bağımsız olarak türetilmiştir. Düzenleyici asgarî ile iyi mühendisliğin vardığı nokta büyük ölçüde örtüşmektedir.

Düşmanca Girdi

İki Ayrı Arıza Eksen

Önceki bölümlerdeki savunma katmanları tek bir arıza eksenine üzerine tasarlanmıştır: *etmenin kendi hatası*. Halüsinasyon gören, tip hatası yapan, sonsuz döngü yazan bir etmen bu katmanlarda yakalanır.

İkinci ve yapısal olarak farklı eksen *düşmanca girdidir*. Bu ekseninde etmen hata yapmaz; talimatı doğru anlar, planı doğru kurar, aracı doğru çağırır — yalnızca aldığı talimat kullanıcıdan gelmemiştir. Statik analiz, birim testi ve tip denetimi hiçbir koruma sağlamaz, çünkü üretilen kod teknik olarak kusursuzdur.

Ayrımın kökeni şudur: klasik yazılımda kod ile veri arasındaki sınır dilin kendisi tarafından çizilir. Dil modellerinde böyle bir sınır yoktur; sistem istemi, kullanıcı mesajı ve araçtan dönen gözlem aynı jeton dizisinin parçalarıdır.

Etmenin okuduğu her şey potansiyel olarak talimattır.

Dolaylı Enjeksiyon

Doğrudan enjeksiyon — kullanıcının kısıtları aşmayı hedefleyen girdisi — etmen sistemlerine özgü değildir. Asıl tehlike dolaylı biçimdir: saldırgan kullanıcıyla hiç temas kurmaz, etmenin *gözlem kanalına* talimat yerleştirir.

1. Etmenin bir kaynağı okuyacağı bilinir
2. Saldirgan o kaynaga gizli talimat yerleştirir
(beyaz metin, HTML yorumu, alt nitelik, belge üst verisi)
3. Etmen içeriği "gözlem" olarak bağlama alır
4. Model bu metni kullanıcı talimatından ayırt edemez
5. Etmen saldirganın hedefini kendi hedefiymiş gibi yurutur

Kritik nokta şudur: etmen bu senaryoda yanlış davranmamaktadır. Döngü tasarımı gereği gözlemi okumak ve planı ona göre güncellemek zorundadır. Saldırı, sistemin hatasını değil **tasarım ilkesinin kendisini** istismar eder.

Saldırı Yüzeyi Envanteri

Saldırı	Mekanizma	Sonuç
Dolaylı enjeksiyon	Gözlem kanalına talimat gömme	Hedefin ele geçirilmesi
Araç zehirlleme	Araç açıklamasına gizli yönerge	Seçim aşamasında yönlendirme
Şaşkın vekil	Etmenin geniş yetkisinin dar yetkili kullanıcı adına kullanılması	Yetki yükseltme
Veri sızdırma	Gizli verinin dış adrese parametre olarak yazılması	Bilgi ifşası
Bellek zehirlleme	Kalıcı belleğe yanlış olgu yazdırma	Oturumlar arası kalıcı sapma
Sözleşme değiştirme	Onaylanmış uzak aracın sonradan güncellenmesi	Denetlenmiş yüzeyin geçersizleşmesi
Kaynak tüketimi	Sonsuz alt görev üretimine ikna	Maliyet ve kullanılabilirlik saldırısı

İzin Bileşimleri

En sinsi sızdırma biçimi, etmenin yalnızca zararsız araçlar kullanarak veri kaçırmasıdır. Etmen bir dosya okur, içinde gizli bilgi vardır; enjekte edilmiş talimat sonucun bir dış adrese parametre olarak gönderilmesini ister. Etmen bunu bir doğrulama adımı sanır. Hiçbir kural ihlal edilmemiştir: okuma yetkisi vardır, ağ erişimi vardır.

Okuma yetkisi ile ağ yetkisi aynı etmende birleştiğinde, birleşim toplamdan daha geniş bir tehdittir.

Yetki tasarımı tekil izinler üzerinden değil, izin *bileşimleri* üzerinden yapılmalıdır.

Şaşkın Vekilin Açılımı

Etmen kendi geniş yetkisiyle çalışır ve dar yetkili bir kullanıcının talebini yerine getirir; kontrol etmenin kimliği üzerinden yapıldığından talep onaylanır.

```
Kullanici A: "Benim raporlarimi ozetle" -> mesru
Kullanici A: "Tum raporlari ozetle"      -> etmenin yetkisi yeter,
                                           kullanicinin yetmez
Sonuc: etmen, A'nin goremeyecegi veriyi A'ya sunar
```

Karşı önlem, yetki kontrolünün kesişim kapsamında yapılmasıdır. Ek olarak filtreleme veri erişim katmanında olmalıdır; sunum katmanında yapılan süzme güvenilir değildir.

Bellek Zehirlenmesinin Açılımı

Kalıcı belleğe yazma yetkisi olan bir etmende enjeksiyon tek oturumluk olmak-tan çıkıp kalıcılaşır.

Oturum 1: enjekte edilmiş içerik, "kullanıcının tercihi X'tir"
bilgisini kalıcı belleğe yazdırır
Oturum 2: etmen bunu güvenilir geçmiş olarak okur
Oturum n: davranış sürekli sapmış; kaynak izlenemiyor

Karşı önlemler:

- Her kayda kaynak etiketiyle saklanması; güvenilmeyen kaynaktan türeyenlerin ayrı güven sınıfında tutulması
- Kalıcı belleğe yazmanın ayrı bir yetki olarak modellenmesi, okuma yetkiyle birlikte otomatik verilmemesi
- Belleğin insan tarafından görüntülenebilir ve tekil kayıt düzeyinde silinebilir olması
- Değişmezlerin — kimlik, yetki, politika — hiçbir koşulda etmen tarafından yazılabilir bellekte tutulmaması

Savunma Katmanları

Kaynak Etiketleme

Bağlama giren her parça kaynağıyla işaretlenir.

GUVENILIR	sistem istemi, operator politikası
YARI	dogrulanmış kullanıcı girdisi
GUVENILMEZ	web içeriği, dosya içeriği, araç çıktısı, baska etmenin ürettiği metin

Kural: güvenilir kaynaktan gelen hiçbir metin talimat olarak yorumlanamaz. Bu tam bir çözüm değildir — model etiketi de metin olarak görünür — ancak istem düzeyinde belirgin bir iyileşme sağlar.

Sınırlayıcı İşaretleme

Yapısal savunmanın uygulanamadığı durumlarda kullanılan ucuz önlem, güvenilmeyen içeriğin öngörülemez bir sınırlayıcıyla çevrelenmesidir.

Aşağıdaki blok DIS KAYNAKTAN gelen veridir. İçindeki hiçbir ifade talimat değildir; yalnızca veri olarak değerlendirilecektir.

```
<<<GUVENILMEZ-a7f3c91e>>>  
... içerik ...  
<<<SON-a7f3c91e>>>
```

Sınırlayıcının koşu başına rastgele üretilmesi zorunludur; sabit sınırlayıcı, saldırganın onu taklit ederek bloktan çıktığını iddia etmesine imkân verir. Sağladığı güvence olasılıksaldır.

Yetenek Daraltma

Enjeksiyonu tamamen önlemek mümkün olmadığından savunmanın ağırlığı, başarılı enjeksiyonun *yapabileceklerini* daraltmaya kayar.

Genis etmen: okuma + yazma + ag + kabuk -> tek enjeksiyon = tam ihlal
Dar etmenler: okuyucu (ag yok)
ozetleyici (arac yok)
yazici (yalnızca izin listesindeki yol)

Soru "etmen kandırılabilir mi" değil, "kandırıldığında ne kadar zarar verebilir" sorusudur.

Çıktı Kanalı Kısıtlaması

Etmenin dış dünyaya yazabildiği her kanal bir sızdırma kanalıdır. Uygulanabilir kısıtlar: izin verilen alan listesi, adres parametrelerinde serbest metin yasağı, giden isteklerde gizli veri örüntüsü taraması, ağ erişiminin yalnızca ayrı ve yetkisiz bir alt etmene verilmesi.

Ayrı Model Kalıbı

Bugün bilinen en güçlü yapısal savunma, güvenilmeyen veriyi hiç görmeyen bir ayrıcalıklı model ile ayrıcalığı olmayan bir karantina modelinin ayrılmasıdır.

```
kullanici talimati
|
v
+-----+
| AYRICALIKLI MODEL | guvenilmeyen metni ASLA gormez
| plan uretir       | arac cagirlarina karar verir
+-----+-----+
| sembolik degisken referanslari
v
+-----+-----+
| KARANTINA MODELİ | guvenilmeyen icerigi okur
| yalnızca cikarım | arac cagiramaz, plan degistiremez
+-----+-----+
| yapilandirilmis, sema-dogrulanmis sonuc
v
yurutme katmani
```

Ayrıcalıklı model karantina modelinin döndürdüğü değeri okumaz; yalnızca değişken olarak taşır. Güvenilmeyen metnin plan üzerindeki etki kanalı yapısal olarak kapanır. Bedeli ifade gücünün daralmasıdır: etmen okuduğu içeriğe göre plan değiştiremez.

Kod Yürütme İzolasyonu

Etmen kod üretip çalıştırıyorsa ayrı bir izolasyon ekseni gerekir. Aşağıdaki matris tek referanstır.

Teknoloji	İzolasyon düzeyi	Not
Yerleşik <code>vm</code>	Bağlam, aynı yığın	Güvenlik mekanizması değildir; yalnızca güvenilen kod, eğitim ve prototip
<code>vm2</code>	Bağlam	Terk edilmiştir; kaçış açıkları nedeniyle bakımı bırakıldı
<code>isolated-vm</code>	V8 izolatu, ayrı yığın	Saf hesaplama için uygun; yerel eklenti bağımlılığı taşır
Süreç izin modeli	Süreç yetenekleri	Dosya ve ağ erişiminin başlangıçta kısıtlanması; ucuz ek katman
Konteyner / <code>gVisor</code>	İşletim sistemi	Dosya sistemi erişimi, paket kurulumu, veritabanı sorgusu
Mikro sanal makine	Hipervizör	Çok kiracılı üretim
WebAssembly	Çalışma zamanı	Determinizm ve taşınabilirlik gerektiren hesaplama

İki nokta vurgulanmalıdır. Birincisi, yerleşik `vm` modülü bir savunma katmanı olarak sunulmamalıdır: ana uygulamayla aynı bellek alanını paylaşır ve prototip zinciri istismarıyla kaçış mümkündür. İkincisi, katmanlar birleştirilebilir; konteyner içindeki bir süreçte ayrıca izin modelinin etkinleştirilmesi, tek katmanın atlatılması hâlinde ikinci sınırı korur.

Genel ilke: *izolasyonun maliyeti, etmenin ihtiyaç duyduğu yan etkinin genişliğiyle doğru orantılıdır*. Yan etkisi olmayan saf hesaplamalar için izolat düzeyi yeterlidir; etmen dış dünyaya yazmaya başladığı andan itibaren sınır işletim sistemi katmanına taşınmalıdır.

Kod Değiştiren Etmenler İçin Ek Katmanlar

Etmen kendi kaynak kodunu veya üretim kodunu değiştiriyorsa aşağıdaki zincir zorunludur.

Katman	İşlevi
Geri alma ağı	Değişiklik öncesi sürüm kaydı; doğrulama başarısızsa anında dönüş. Hatayı önlemez, <i>kalıcı</i> olmasını önler
Statik analiz	Derleyici ve tip denetimi; kod çalıştırılmadan hatanın gözlem olarak etmene iadesi
İzole yürütme	Ağ ve disk erişimi kısıtlı geçici ortamda çalıştırma
Test odaklı doğrulama	Hatayı yeniden üreten testin yazılması ve izole ortamda koşturulması
Onay kapısı	Dosyayı ezmek yerine inceleme talebi açma; geri alınamaz son adımın insana bırakılması

Çalışan sürece doğrudan kod enjekte eden sıcak yama teknikleri bu zincirin dışındadır ve kullanılmamalıdır; aynı bölümdeki izolasyon ilkelerinin en uç ihlali-
lidir. Sıcak güncelleme gerekiyorsa süreç yeniden başlatma veya modül yeniden
yükleme kullanılır.

Mimari Duruş

Prompt enjeksiyonu çözülmüş bir problem değildir. Hiçbir istem savunması, sınıflandırıcı veya kaçış şeması tam güvence sağlamaz. Doğru duruş şudur: enjeksiyonun *gerçekleşeceği* varsayılır ve sistem, gerçekleştiğinde etkinin sınırlı kaldığı biçimde tasarlanır.

Arıza Kipleri, Entropi ve Öğrenme

Katman Bazlı Arıza Envanteri

Muhakeme Katmanı

Arıza kipi	Belirti	Karşı önlem
Uydurma	Var olmayan API, dosya veya alan üretimi	Şema doğrulama, varlık kontrolü, kısıtlı çözümleme
Hedef kayması	Ara adımın nihai hedefin yerine geçmesi	Hedef değişmezliği, açık başarı ölçütü
Erken sonlandırma	Kısmi çözümün tamamlanmış sayılması	Bağımsız doğrulama, açık tabanlı döngü
Doğrulama yanlışlığı	İlk hipotezin bağlamı kirlenmesi	Taze bağlamalı alt etmen
Aşırı ayrıştırma	Basit işin gereksiz alt etmene devri	Yapısal taban kuralı

Araç Katmanı

Arıza kipi	Belirti	Karşı önlem
Yanlış araç seçimi	Anlamsal olarak yakın araçlardan yanlışın çağrılması	Olumsuz alan tanımı, "ne zaman" açıklaması
Argüman uydurma	Zorunlu parametrenin makul görünen değerle doldurulması	Katı şema, boş değer yasağı
Sessiz başarısızlık	Aracın hata yerine boş sonuç dönmesi	Açık hata sözleşmesi
Etkisiz tekrar	Aynı çağrının aynı argümanla yinelenmesi	Çağrı imzası önbellegi
Yan etki yinelenmesi	Yeniden denemede işlemin ikinci kez uygulanması	İdempotans anahtarı

Bellek ve Durum Katmanı

Arıza kipi	Belirti	Karşı önlem
Bağlam taşması	Erken mesajların sessizce düşmesi	Kayan pencere, özetleme, sistem istemi koruması
Özet bozulması	Özetleme sırasında kritik kısıtın kaybı	Değişmezlerin özet dışında tutulması
Bayat durum	Dış dünya değişmişken eski gözlemle karar	Gözlem tazeliği damgası
Defter çatışması	Paralel etmenlerin aynı kaydı ezmesi	Ekleme-yalnız defter, iyimser kilit

Koordinasyon Katmanı

Arıza kipi	Belirti	Karşı önlem
Sonsuz pas atma	İki etmenin işi karşılıklı devretmesi	Devir sayacı, sahiplik zorunluluğu
Sorumluluk boşluğu	Hiçbir etmenin adımı üstlenmemesi	Tek sahip kuralı
Yankı odası	Etmenlerin birbirinin hatasını doğrulaması	Temiz oda ayrımı
Mesaj patlaması	Etmen sayısı ile iletişimin karesel büyümesi	Yıldız topolojisi, paylaşılan defter
Kaynak açlığı	Bir etmenin bütçenin tamamını tüketmesi	Etmen başına kota

Ağırlıklandırma İlkesi

FMEA'nın klasik önceliklendirmesi şiddet, olasılık ve tespit edilebilirlik çarpımıdır. Etmen sistemlerinde üçüncü çarpan belirleyicidir: sessiz arızalar — sessiz başarısızlık, bayat durum, yankı odası — gürültülü arızalardan çok daha tehlikelidir.

Gürültülü arıza gecikme üretir; sessiz arıza yanlış sonuç üretir.

Gürültülü arıza döngüyü durdurur; sessiz arıza döngüyü yanlış yönde ilerletir. Tasarım önceliği bu nedenle hatayı önlemekten çok, hatayı *gürültülü hâle getirmektir*.

Entropi

Etmen sistemlerinin karşısındaki gerçek düşman yetersizlik değil entropidir. Sistem büyüdükçe doğal olarak oluşan bozulmalar:

baglam sismesi	kural sismesi
yetki sismesi	arac sismesi
bellek sismesi	is akisi sismesi
etmen sismesi	bilgi yinelenmesi
celisen talimatlar	bayat durum
izlenemeyen kararlar	

Komutlar zamanla örtüşür ve yönlendirme belirsizliği yaratır. Birbirine yakın adlı dört komut, etmenin hangisini seçeceğini belirsizleştirir ve seçim hatası oranını artırır.

Tek tek mantıklı kurallar birlikte tutarsız politika üretebilir:

Kural A: kısa cevap ver

Kural B: bütün kenar durumları açıklayarak anlat

Kural C: kullanıcıya soru sorma

Kural D: belirsizlik varsa açıklama iste

Talimat kümesi büyüdüğünde bir politika motoru problemi doğar: çatışma tespiti, öncelik ve kapsam gerekir.

Dolayısıyla etmen mimarisinin önemli bir işlevi yaratma değil *temizliktir*. Periyodik konfigürasyon denetiminin başarısı daha fazla kural bulmak değil, çoğu zaman silinecek şey bulmaktır.

Artefaktlar Yazılımdır

Sistem davranışının önemli bir kısmı artık kurallarda, istemlerde, komutlarda, yeteneklerde ve yönlendirme yapılandırmasında yaşar. Bunlardan biri bozulduğunda davranış bozulur ama derleyici hiçbir şey söylemez.

Doğal dildeki yapılandırma da yazılımdır.

Gerektirdikleri: sürümleme, test, biçim denetimi, çatışma tespiti, inceleme, gözlemlenebilirlik. Çıktı olasılıksal olsa bile değişmezler test edilebilir.

Taşıyıcı Metin

Mimaride taşıyıcı duvar vardır: sıradan duvar gibi görünür, kaldırılırsa bina çöker. Talimat metinlerinde de öyledir. Uzun bir yetenek dosyasının büyük kısmı silindiğinde performans korunabilirken, birkaç küçük uyarı cümlesi silindiğinde sistem çökebilir.

Bu cümleler yılların arıza bilgisini sıkıştırmıştır — örtük bilginin tek satıra inmiş hâli. Sıradan dokümanda bir uyarı önemsiz ayrıntıdır; etmen dokümanında *yürütülebilir bilgidir*.

Test yöntemi ablasyondur: sil, sistemi çalıştır, kalite düşüyorsa taşıyıcıdır. İstem mühendisliği böylece ampirik mühendisliğe dönüşür. Ölçüt kelime sayısı değil marjinal davranışsal değerdir.

Artefakt Dağıtım Hattı

degisiklik -> sema ve bicim kontrolu -> altin ornek degerlendirmesi
-> kiyaslama (mevcut surum vs aday)
-> kanarya (trafigin kucuk bir kismi)
-> metrik esigi tutuyorsa kademeli yayim
-> tutmuyorsa otomatik geri alma

Kritik nokta geri alma birimidir. İstem sürümü, model sürümü ve araç şeması birlikte sürümlenmelidir; yalnızca birinin geri alınması tutarsız bir bileşim üretir.

Üçlüyü birlikte dağıt: istem sürümü + model sabiti + araç şeması sürümü.

Sistem Düzeyinde Öğrenme

Öğrenme türü	Değişen
Model öğrenmesi	Ağırlıklar
Bağlam içi öğrenme	Oturum içi bağlam
Sistem öğrenmesi	Sistemin dış artefaktları

Etmen sistemlerinde üçüncüsü en güçlü kaldıraçtır ve tek kalıcı olanıdır.

ariza gozlendi -> neden oldu? -> genellenebilir ders cikar
-> kural / yetenek / test / komut uret
-> bir sonraki etmen bunu baslangictan bilir

Örnek: sistem tekrar tekrar aynı doğrulama adımını atlıyorsa, normal yaklaşım "bir daha dikkat et" demektir; etmen yaklaşımı bir doğrulayıcı üretmektir.

Bilgi → yürütülebilir kısıt

Bu dönüşüm gerçekleşmediğinde "bunu geçen sene de yaşamıştık" denir. Gerçek kurumsal öğrenmede sistem, aynı hata sınıfının tekrarını yapısal olarak zorlaştırır.

Artefakt üretimi otomatikleştirilebilir ama onaylanamaz: sinyaller toplanır (kural geçersiz kılma oranı, komut başarısızlığı, tekrarlanan manuel eylem), eşik aşıldığında yeni artefakt *önerilir*. Sistem kendi kritik kurallarını sessizce değiştirmez; insan incelemesi zorunludur.

Kayıt–Tekrar

Tekrarlanabilirlik altyapı desteği olmadan sağlanamaz.

Kayıt modu: her model cagrısı, her arac cagrısı ve her donus degeri imzalanip diske yazilir

Tekrar modu: cagrilar gercekten yapilmaz, kayittan servis edilir

Bu yapı üç şeyi mümkün kılar: döngü mantığındaki hataların model maliyeti olmadan ayıklanması, gerileme testlerinin deterministik koşması, bir üretim arızasının birebir yeniden üretilmesi. Defter kararları kaydeder; kayıt–tekrar girdileri de kaydeder.

Arayüz ve Protokol Katmanı

Arayüzün Tüketicisi Değişti

Klasik arayüzler bir insanın dokümantasyonu okuyacağını, entegrasyon kodunu yazacağını ve bakımını üstleneceğini varsayıyordu. Artık tüketici doğrudan bir etmen olabilir.

ESKI: A API -> dokuman -> geliştirici -> adaptor -> B API
YENİ: A yetenekleri -> etmen keşfeder -> anlar
-> birleştirir -> B yetenekleri

Bu, statik API sözleşmelerinden çalışma anında keşfedilebilir, kendini tarif eden yetenek yüzeylerine geçiştir.

Yetenek Öncelikli Mimari

Ürünü ekranlarından değil yapabildiği işlerden düşünmek:

Ekran odaklı	Yetenek odaklı
Giriş ekranı, gösterge paneli, arama sayfası	kimlik doğrula, ara, sipariş oluştur, sipariş iptal et, fatura getir

Arayüz ürünün kendisi değil, ürünün başlarından biridir; web, mobil, ses ve etmen aynı yetenek katmanının farklı başlarıdır.

Anlamsal Sözleşme

Etmen için bir uç nokta adresi yetmez. Gereken:

”Aktif ticari ilişkisi bulunmayan müşteriye arşivlemek için kullan.
Geçmiş kayıtları silmez. Geri alınabilir. Ödenmemiş fatura varsa kullanma.”

Etmen yüzeyinin gerçek bileşimi: **şema + anlambilim + kısıtlar + uygulanabilirlik**.

Bu, dokümantasyonun statüsünü değiştirir. ”Bu uç nokta yalnızca kesinleşmiş fatura üzerinde kullanılabilir” cümlesi insan için açıklayıcı bir nottur; etmen için bir karar sınırır. Dokümantasyon açıklayıcı artefakt olmaktan çıkıp *operasyonel artefakt* hâline gelir.

Ne Zaman’ı Tarif Et

Araç açıklamasının işi kullanım kılavuzu olmak değil, yönlendirme sinyali ver-mektir.

Zayıf	Güçlü
kullaniciAra: Kullanıcı veritabanında arama yapar ve kullanıcıları döndürür.	Mevcut bir kullanıcıyı kimlik nitelikleriyle bulmak için kullan; kullanıcı oluşturma veya değiştirme için kullanma.

Etmen zaten adından işlevi çıkarabilir; asıl belirsizlik *hangi durumda seçmeliyim* sorusundadır. Açıklama bir anlamsal sevk meta-verisidir.

Olumsuz Alan

Bir şeyi tarif etmek yalnızca ne olduğunu değil, ne olmadığını da söylemektir.

Nesne	Olumsuz alan
Araç	Şu durumda kullanma
Yetenek	Şu problem için uygun değildir
Etmen	Şu sınırı geçemez
Başarı	Şu durumda tamamlanmış sayılmaz

Olumsuz alan, karar sınırını çizdiği için orantısız bilgi taşır.

Araç Tanecikliği

Az sayıda "süper araç" her zaman kolay değildir; bir araç çok sayıda koşullu parametre taşıyorsa seçim ve argüman oluşturma zorlaşır. Odaklı işlemler, geniş kapsamlı araçlardan daha güvenilir çağırılır.

Az parametre, az araçtan iyidir.

Ayrıca bir yeteneğin içeriği kusursuz olsa bile keşif açıklaması kötüyse hiç çağırılmaz. Varlık ile keşfedilebilirlik ayrı kalite boyutlarıdır.

Keşif Protokolü

Bir keşif protokolü (Model Context Protocol örneğinde olduğu gibi), etmenin dış araç, veri kaynağı ve istem şablonlarını çalışma zamanında keşfedip kullanabilmesi için tanımlanmış açık bir istemci-sunucu protokolüdür.

Bileşen	İşlev
Araçlar	Etmenin çağırabileceği, yan etkisi olan işlemler
Kaynaklar	Etmenin okuyabileceği veri kaynakları
İstemler	Sunucunun sağladığı yeniden kullanılabilir şablonlar
Örnekleme	Sunucunun istemciden model çıkarımı talep etmesi

Mimari katkısı, her etmen–araç çifti için ayrı adaptör yazma zorunluluğunu kaldırmasıdır: M etmen ve N araç için $M \times N$ entegrasyon yerine $M + N$ uyarlama yeterli olur.

Protokolün Devretmediği Sorumluluklar

Protokol yalnızca "neler mevcut" sorusunu cevaplar. Sağlamadığı güvenceler:

- Araç açıklamasının dürüst olduğu — araç zehirlenme yüzeyi açık kalır
- Sunucunun sonradan değişmeyeceği — sözleşme değiştirme yüzeyi açık kalır
- Etmenin o aracı kullanmaya yetkili olduğu — yetkilendirme ayrı katmandır
- İşlemin geri alınabilir olduğu — yan etki semantiği araç tarafından

Üretimde protokol ile etmen arasına bir politika geçidi konulması bu nedenle zorunludur: sunucunun sunduğu yüzey ile etmene açılan yüzey aynı olmamalıdır.

Sorumluluk Ayrımı

Katman	Sorusu
Keşif protokolü	Neler mevcut?
Etmen / muhakeme	Hangisini kullanmalıyım?
Politika geçidi	Hangisini kullanmana izin var?
Altındaki sistem	Gerçek işlemi gerçekleştir

Keşif ≠ muhakeme ≠ yetkilendirme ≠ yürütme

Kademeli Açığa Çıkarma

Çok sayıda araç şeması bağlamı şişirir, maliyet ve doğruluk sorunları doğurur. Keşif, her şeyi önceden yüklemek değildir:

once ihtiyaci anla -> ilgili yetenek alanini kesfet
-> gereken araclari yukle -> kullan

Etmen–Etmen Ekseni

Etmen–araç ekseninin yanında etmen–etmen ekseni farklı problemler doğurur: kimlik doğrulama, görev devri semantiği, kısmi sonuç aktarımı, iptal ve zaman aşımı yayılımı, sorumluluk zinciri.

Arac protokolü : "Ne yapabilirim?" -> yetenek kesfi
Etmen protokolü: "Kim bu işi üstlendi?" -> görev ve sorumluluk devri

İkinci ekseninde olgunluk düşüktür; bu nedenle çoklu etmen sistemlerinde koordinasyonun büyük kısmı hâlâ paylaşılan defter ve uygulamaya özgü sözleşmelerle yürütülmektedir. Klasik iletişim dilleri bölümünde belirtildiği gibi, tipli mesajlara dönüş beklenebilir.

Uzun Koşularda Kullanıcı Etkileşimi

Dakikalarca süren bir koşu, tek türlü bir sohbetten farklı bir etkileşim tasarımı gerektirir.

Gereklilik	Gerekçe
İlerleme görünürlüğü	Sessiz bekleme, sistemin kilitlendiği izlenimi verir
Adım düzeyinde şeffaflık	Hangi aracın hangi argümanla çağrıldığının görülebilmesi
İptal	Yanlış yöne giden koşunun bütçe tükenmeden durdurulabilmesi
Kısmi sonuç	İptal veya zaman aşımında o ana kadarki çıktının teslimi
Yönlendirme	Koşu sürerken düzeltici girdi verilebilmesi
Devam ettirme	Kesintiye uğrayan koşunun defterden sürdürülebilmesi

İptal ve yönlendirme düzeltilebilirlik bölümüyle doğrudan bağlantılıdır: ikisi de etmenin yorumuna bırakılmamalı, yürütme katmanında uygulanmalıdır.

Şeffaflık, ham düşünce zincirinin kullanıcıya dökülmesi anlamına gelmez. Ham iz üç sorun üretir: gürültü, güvenilmeyen içeriğin kullanıcıya aracısız aktarılması, terk edilmiş hipotezlerin sonuç sanılması. Doğru biçim defterin özetlenmiş görünümüdür:

[1] Depo tarandi	12 dosya
[2] Hipotez: onbellek gecersizlemesi	(elendi)
[3] auth/session.ts incelendi	satir 87
[4] Test yazildi	1 basarisiz
[5] Duzeltme uygulandi	
[6] Test kosuldu	3 basarili

Belirlenim Sınırı

Model Nerede, Kod Nerede

Model kullan	Deterministik kod kullan
Anlamsal belirsizlik	Kesin iş kuralı
Bulanık sınıflandırma	Tam tutarlılık gereksinimi
Açık uçlu dil yorumu	Çok düşük gecikme
Niyet çıkarımı, anlam üretimi	Doğrulama, birleştirme, dağıtım, sözleşme dayatma

Belirsizliği modele, kesinliği deterministik sisteme ver.

Bir yaş eşiği kontrolü için model kullanmanın anlamı yoktur. Buna karşılık "bu müşteri mesajı mevcut politika kapsamında istisna gerektiriyor mu" sorusu muhakeme gerektirir.

Olasılıksal Bileşen Güvence Modelini Değiştirir

Derleyici aynı girdiye kararlı çıktı verir; model olasılıksal yorum üretir. Sistem düzeyindeki sonucu şudur: model hattında şema doğrulama, şablon çıpalama, yeniden deneme ve anlamsal doğrulama daha önemlidir. Olasılıksal bileşen eklemek yalnızca bir bileşen değişimi değildir; *tüm sistemin güvence modelini değiştirir*.

Bulanıktan Yapılandırılmış

bulanik girdi -> anlamsal cikarim -> yapilandirilmis alanlar
-> deterministik alt akis

Model sistemin tamamı yerine *belirsizlik sınırında* kullanılır.

Kısıtlı Çözümleme

Uydurma ve argüman uydurma kiplerinin en etkili karşı önlemi kısıtlı çözümlemedir: model çıktısı, örnekleme aşamasında bir dilbilgisine veya şemaya zorlanır; şemayı ihlal eden jetonlar hiç üretilmez.

Katman	Garantisi
İstemde "JSON üret"	Hiçbiri; olasılıksal uyum
Şema doğrulama (son- radan)	Bozuk çıktı yakalanır, ancak koşu boşa gider
Kısıtlı çözümleme	Sözdizimsel geçerlilik yapısal olarak garanti

Kritik sınır: kısıtlı çözümleme sözdizimini garanti eder, anlamı garanti etmez. Şemaya uygun ama var olmayan bir dosya yolu üretilir. Biçim doğrulama ile anlam doğrulama ayrımı burada da geçerlidir.

Yan Etki Semantiği

Yeniden deneme mantığı, aracın yan etki sınıfını bilmeden çalıştırılmaz.

Araç sınıfı	Yeniden deneme politikası
Saf okuma	Serbestçe yeniden denenebilir
İdempotent yazma	Aynı anahtarla yeniden denenebilir
İdempotent olmayan yazma	Yalnızca sonuç sorgulanabiliyorsa denenebilir
Geri alnamaz	Otomatik yeniden deneme yasak; yükseltme zorunlu

Bu sınıf araç şemasının bir parçası olmalıdır:

```
{  
  "name": "faturaOlustur",  
  "yanEtki": "yazma",  
  "idempotent": true,  
  "idempotansAnahtariParametresi": "istekId",  
  "geriAlinabilir": false,  
  "onayGerektirir": true  
}
```

Bu meta-veri, anlamsal sözleşmenin makinece işlenebilir hâlidir: yalnızca etmenin okuması için değil, yürütme katmanının politika uygulaması için gereklidir.

Dayanıklılık Asgarisi

Önlem	İşlevi
Çağrı zaman aşımı	Tek aracın döngüyü süresiz askıya almasını engeller
Üstel geri çekilme	Hız sınırına takılan çağrının sistemi kilitlemesini önler
Titreşim	Paralel etmenlerin eşzamanlı yeniden denemesini dağıtır
Devre kesici	Sürekli başarısız aracı geçici olarak devre dışı bırakır
Bütçe sayacı	Koşu başına jeton, süre ve alt etmen üst sınırı

Bütçe sayacı, yineleme sınırının genelleştirilmiş hâlidir. Yineleme sayısı tek başına yetersizdir: tek bir yinelemede çok sayıda alt etmen açan bir etmen, birkaç adımda bütçeyi tüketir.

Sonuç

Bu çalışma, etmen sistemlerini bir uygulama tekniği olarak değil bir karar sistemi mühendisliği problemi olarak ele almıştır. Beş sonuç öne çıkmaktadır.

Birincisi, etmen düşüncesi yeni bir kavram değildir. Bandura'nın faillik modeli, Simon'ın sınırlı rasyonalitesi, Boyd'un döngüsü, BDI'nin taahhüt kavramı, Contract Net'in tahsis protokolü ve Brooks'un yerel yeterlilik ilkesi bugünkü tartışmanın büyük kısmını önceler. Dil modeli bu döngüyü ucuzlatan bir motordur; döngünün kendisi değildir. Bu tespitin pratik değeri, çözülmüş problemlerin yeniden çözülmesini önlemesidir.

İkincisi, etmen mimarisi varsayılan değil istisnai bir seçimdir. Üç koşullu ölçütün en belirleyicisi üçüncüsüdür: başarı dış kanıtla doğrulanabiliyor mu. Doğrulanamayan bir görevde etmen mimarisi hata oranını düşürmez, hatanın tespit edilebilirliğini düşürür. Kademeli açılma merdiveni — tek çağrıdan alt etmenli sisteme — bir stil tercihi değil, entropiye karşı önleyici tedbirdir.

Üçüncüsü, güvenilirlik modelde değil döngüdedir. Etmen kalitesini belirleyen şey ne zaman durduğu, neyi hatırladığı, sonucu kimin kontrol ettiği ve hata olunca ne yaptığıdır. Daha güçlü model, eksik bir doğrulama katmanının yerine geçmez. Bellek modelin içinde değil çağırın sistemin içindedir; bu nedenle bağlam ve bellek tasarımı bir optimizasyon detayı değil çekirdek mimari karardır.

Dördüncüsü, esneklik ile saldırı yüzeyi aynı yapısal özelliğin iki adıdır. Gözlemin planı değiştirebilmesi ReAct'in gücüdür ve tam olarak dolaylı enjeksiyonun kullandığı kanaldır. Bu nedenle mimari seçimi bir performans kararı olduğu kadar bir güvenlik kararıdır ve enjeksiyonun gerçekleşeceği varsayımıyla tasarım yapmak, önlemeye çalışmaktan daha sağlam bir duruştur.

Beşincisi, asıl tehlike gürültülü arıza değil sessiz arızadır. Çöken bir etmen döngüyü durdurur; sessizce yanlış yönde ilerleyen bir etmen kaynağı ve güveni birlikte tüketir. Bu nedenle tasarımın önceliği hatayı önlemekten çok hatayı fark edilebilir kılmaktır: bağımsız doğrulama, olay defteri, kaynak zinciri ve öz-beyana güvenmeme ilkesi hep bu tek amaca hizmet eder.

Nihai olarak, otonomi mühendisin rolünü ortadan kaldırmaz. Onu *kod yazar* konumundan *sınır çizen* konumuna taşır ve bu ikinci iş, birincisinden daha az teknik değildir.

Ek A: Node.js ile Uygulama

Bu Ekin Statüsü

Bu ek, gövdedeki ilkelerin belirli bir ekosistemdeki karşılıklarını gösterir. Kod sürüm bağımlıdır ve hızla bayatlar; gövdedeki hiçbir iddia buradaki örneklere dayanmaz. Örnekler doğrudan üretime taşınacak biçimde değil, tartışılan kısıtları gösterecek biçimde yazılmıştır ve taklit uygulamalar açıkça işaretlenmiştir.

Araç Kaydı ve Şemalar

Araç kaydı, argümanları nesne olarak alacak biçimde tanımlanır. Argümanları konumsal olarak açmak (...Object.values(args)) kırılığandır: modelin ürettiği JSON'da anahtar sırası garanti değildir ve iki parametrelili bir fonksiyonda sessizce yanlış eşleşme oluşur.

```
// TAKLIT UYGULAMA - gerçek servis çağrısı yapmaz.
// Üretimde bu fonksiyonlar dışarıdaki servise gider.
const toolRegistry = {
  getWeather: async ({ location }: { location: string }) => {
    return '[MOCK] ${location}: yağmurlu, 15 derece';
  },
  getQuestion: async ({ category }: { category: string }) => {
    return '[MOCK] ${category} kategorisinde örnek soru';
  }
};

const toolDefinitions = [
  {
    type: "function",
    function: {
      name: "getWeather",
      // "Ne zaman"i tarif et: yönlendirme sinyali
      description:
        "Belirli bir şehrin hava durumunu getirmek için kullan. " +
        "Genel iklim bilgisi veya geçmiş veri için kullanma.",
      parameters: {
        type: "object",
        properties: {
          location: { type: "string", description: "Şehir adı" }
        },
        required: ["location"],
        additionalProperties: false
      }
    },
    // Yan etki semantigi: yurutme katmanı için
    sideEffect: "none",
  },
];
```

```

    idempotent: true,
    reversible: true
  }
];

```

Bütçe Sınırlı Döngü

Yineleme sınırı yalnızca ilan edilmez, kodlanır. Sınır aşıldığında sessizce dönmek yerine hata fırlatmak zorunludur; sessiz kesme, arıza tablosundaki *erken sonlandırma* kipini üretir.

```

const budget = {
  maxIterations: 10,
  maxTokens: 200_000,
  maxWallClockMs: 300_000,
  maxSubagents: 8,
  maxToolCalls: 60
};

async function runAgent(messages: any[]) {
  let iteration = 0;
  const startedAt = Date.now();

  while (true) {
    if (++iteration > budget.maxIterations) {
      throw new Error(
        `Yineleme siniri asildi (${budget.maxIterations}).`
      );
    }
    if (Date.now() - startedAt > budget.maxWallClockMs) {
      throw new Error("Sure butcesi asildi.");
    }

    const response = await client.chat.completions.create({
      model: PINNED_MODEL_ID, // surum damgali kimlik
      temperature: 0, // arac secimi icin dusuk sicaklik
      seed: RUN_SEED, // desteklendiginde tekrarlanabilirlik
      messages,
      tools: toolDefinitions,
      tool_choice: "auto"
    });

    const msg = response.choices[0].message;
    messages.push(msg);

    if (!msg.tool_calls) {

```

```

// Model bitti diyor. Bu KANIT DEGILDIR.
// Dis dogrulama burada cagrilir.
const verdict = await independentVerify(msg.content);
if (verdict.pass) return msg.content;

messages.push({
  role: "user",
  content: 'Dogrulama basarisiz. Kalan acik: ${verdict.gap}'
});
continue;
}

await executeToolCalls(msg.tool_calls, messages);
}
}

```

Dikkat edilmesi gereken nokta, döngünün modelin beyanıyla değil doğrulayıcının kararıyla sonlanmasıdır. Başarısızlıkta geri dönen bilgi "tekrar dene" değil, kalan *açığın* tarifidir.

Paralel Araç Çağrısı ve Kısmi Başarısızlık

Modern API'ler tek turda birden çok bağımsız çağrı döndürebilir. Paralel yürütme gecikmeyi azaltır ama kısmi başarısızlık problemini doğurur.

```

async function executeToolCalls(toolCalls: any[], messages: any[]) {
  const results = await Promise.allSettled(
    toolCalls.map(async (call) => {
      const handler = toolRegistry[call.function.name];
      if (!handler) {
        // Uydurma arizasi: istisna yerine gozlem olarak geri ver
        return { call, error: 'Bilinmeyen arac: ${call.function.name}' };
      }
      const args = JSON.parse(call.function.arguments);
      return { call, value: await handler(args) };
    })
  );

  for (let i = 0; i < results.length; i++) {
    const r = results[i];
    const call = toolCalls[i];
    const payload = r.status === "fulfilled"
      ? r.value
      : { error: String(r.reason) };

    // HER tool_call_id icin mutlaka bir yanit eklenmelidir
  }
}

```

```

    messages.push({
      tool_call_id: call.id,
      role: "tool",
      name: call.function.name,
      content: payload.error
        ? 'HATA: ${payload.error}'
        : String(payload.value)
    });
  }
}

```

İki kural zorunludur. Birincisi `Promise.allSettled` kullanılmalıdır; tek aracın hatası diğerlerinin sonucunu düşürmemelidir. İkincisi her çağrı kimliği için bir yanıt mesajı eklenmelidir; eksik bırakılan bir kimlik sonraki çağrıda protokol hatası üretir. Bu, pratikte en sık karşılaşılan kırılma noktasıdır.

Paralel yürütmenin ön koşulu çağrılarının bağımsızlığıdır. Yan etkili araçlarda sıra bağımlılığı varsa paralelleştirme sessiz veri bozulması üretir; araç tanımına sıra duyarlılığı meta-verisi eklenmelidir.

Kalıcı Durum Şeması

Görüntüleme belleği ile çalışma belleği ayrı alanlarda tutulur. Belge tabanlı bir depoda dizi alanlarında birleştirme kısmi çalışmaz; diziyi bütünüyle değiştirir. Bu nedenle iki alan ayrı yazılır.

```

chats/{sessionId}
  displayHistory : [] // arayuz icin, hic kirpilmaz
  contextState   : [] // modele giden, optimize edilen calisma bellegi
  summary        : "" // sikistirilmis gecmis, kaynak kayitlara bagli
  invariants     : [] // asla ozetlenmez, asla kirpilmaz
  updatedAt      : ts

```

Olay defteri ayrı bir koleksiyonda ekleme-yalnız tutulur:

```

runs/{runId}/events/{seq}
  type      : "tool_called" | "verification_failed" | ...
  payload   : {...}
  agentId   : "..."
  modelPin  : "..."
  policyVer : "..."
  at        : ts

```

Bağlam Optimizasyonu

Kayan pencere en basit stratejidir. Sistem istemi ve değişmezler pencerenin dışında tutulur.


```
function slidingWindow(all: any[], windowSize = 6) {
  const pinned = all.filter(m => m.role === "system");
  const rest = all.filter(m => m.role !== "system");
  return [...pinned, ...rest.slice(-windowSize)];
}
```

Özetleme kullanılıyorsa özetin kaynak kayıtlara bağlanabilir tutulması gerekir; aksi hâlde silme hakkı yükümlülüğü karşılanamaz ve özet içindeki bir kısıtın kaybı sessizce gerçekleşir.

İzolasyon Notu

Yerleşik `vm` modülü ana uygulamayla aynı bellek alanını paylaşır ve resmî dokümantasyonda güvenlik mekanizması olmadığı açıkça belirtilir. Prototip zinciri istismarıyla kaçış mümkündür. Bu nedenle örnek kod yalnızca zaman aşımı davranışını göstermek için verilir, güvenilmeyen kod için kullanılmaz.

```
// YALNIZCA GUVENILEN KOD ICIN. Guvenlik siniri degildir.
const vm = require("vm");
const sandbox = { data: { x: 10, y: 20 } };
vm.createContext(sandbox);

try {
  const result = vm.runInContext("data.x + data.y", sandbox, {
    timeout: 500
  });
  console.log(result);
} catch (err) {
  console.error("Sandbox hatasi:", err.message);
}
```

Güvenilmeyen kod için ayrı yığına sahip izolat, süreç izin modeli veya konteyner düzeyi izolasyon kullanılır; seçim, gövde bölümündeki matrise göre yapılır.

Ek B: İlkeler Envanteri

İlke	İçerik
Hedef > yordam	Kalıcı olan ne ve neden; nasıl yeniden çözülebilir
Otonomi = sınırlı karar hakkı	Sınırsızlık değil, belirli uzayda karar yetkisi
Yetenek $\neq$ yetki	Araca erişim, kullanma izni değildir
Güven $\neq$ onay	Eminlik izin üretmez
Bellek $\neq$ bağlam penceresi	Kalıcı olgular dışarıda, konuşma geçici
Öz-beyan $\neq$ kanıt	Tamam diyene değil dünyanın durumuna bak
Üretim $\neq$ doğrulama	Üretici ve hakem epistemik olarak ayrı
Bileşen doğrulandı $\neq$ yetenek kanıtlandı	Uçtan uca kanıt gerekir
Biçim doğrulama $\neq$ anlam doğrulama	Güzel görünen yanlış cezalandırılmalı
Yerel doğruluk $\neq$ küresel başarı	Her parça doğru, bütün yanlış olabilir
Keşif $\neq$ muhakeme $\neq$ yetkilendirme $\neq$ yürütme	Dört ayrı sorumluluk
Doğal dil yapılandırma = yazılım	Test, sürüm, denetim gerektirir
Bağlam = bütçe	Sınırlı kaynak; doğru anda minimum yeterli
Bilgi yaşam döngüsü $\neq$ kod yaşam döngüsü	Bilgiyi dağıtım birimine bağlama
Bireysel faillik $\neq$ kolektif zekâ	Sürü ayrıca koordinasyon mimarisi ister
Süreç atılabilir, durum kalıcı	Süreç ölebilir, görev ölmemeli
Açığa göre döngü	Deneme sayısı başarı ölçütü değildir
Etmen kalitesi $\neq$ model kalitesi	Güvenilirlik döngüde, modelde değil
Az parametre > az araç	Tanecikliği parametre yüküne göre ayarla
Ne zaman'ı tarif et	Açıklama yönlendirme meta-verisidir
Mümkünse deterministik	Belirsizlik yoksa muhakeme motoru kullanma
Otonomi sonuçla ölçeklenir	Otonomi ikili değil, riskle kademeli
İnsan = yükseltme hedefi	İnsanı her adıma koyma
Niyeti bağla, hareketi değil	Niyet sürümlenir, bağlama atılabilir
Talimatlar modelden uzun yaşamalı	Yöntemi değil değişmezi kodla
Arıza $\rightarrow$ yürütülebilir kısıt	Ders doğrulayıcıya dönüşmeli
Dünyayı sorgula, serileştirme	Bağlam şişmesinin büyük kısmı gereksiz kopyadır
Durdurma bir yürütme özelliğidir	İptal etmenin yorumuna bırakılmaz
Esneklik = saldırı yüzeyi	Aynı özelliğin iki adı
Sessiz arıza gürültüden tehlikelidir	Öncelik, hatayı gürültülü kılmaktır
Entropi asıl düşmandır	Mimarinin işi yaratma değil temizlik

Ek C: Sözlük

Terim	Tanım
Etmen	Hedef doğrultusunda gözlem yapan, kendi ara kararlarını üreten, eyleme geçen ve geri bildirimle düzelen sistem
ReAct	Düşünce ve eylemin dönüşümlü ilerlediği döngü mimarisi
Planla-ve-Yürüt	Planın gözlemden önce üretilip sabitlendiği mimari
Zincirleme düşünme	Ara muhakeme adımlarının açıkça üretilmesi
Düşünce Ağacı	Birden çok muhakeme dalının açılıp budandığı arama
Araç çağırma	Modele araç şemaları sunularak dış fonksiyon çağırısı yaptırılması
Bağlam penceresi	Modelin tek seferde işleyebileceği jeton sınırı
Kayan pencere	Yalnızca son N mesajın gönderildiği bellek stratejisi
Geri getirmeli üretim	Vektör aramasıyla ilgili belleğin çağırılması
Yönetici kalıbı	Merkezî bir etmenin görev dağıtıp sonuçları birleştirmesi
Blackboard	Etmenlerin ortak bir durum nesnesi üzerinden çalıştığı kalıp
BDI	İnanç, arzu ve niyeti ayrı depolarda modelleyen klasik etmen mimarisi
Contract Net	Görevin ilan, teklif ve tahsisle dağıtıldığı koordinasyon protokolü
Subsumption	Merkezî model yerine katmanlı tepkisel davranışlara dayanan mimari
POMDP	Kısmi gözlemlenebilir Markov karar süreci
İnanç durumu	Etmenin gözlemlerden türettiği, gerçek durum üzerindeki dağılım
Şartname istismarı	Ölçütü sağlayıp niyeti ıskalayan optimizasyon davranışı
Ödül istismarı	Ödül sinyalinin hedefi gerçekleştirmeden yükseltilmesi
Düzeltilbilirlik	Sistemin durdurulmaya ve düzeltilmeye direnmemesi
Araçsal yakınsama	Farklı nihai hedeflerin benzer ara hedeflere yönelmesi
Şaşkın vekil	Yüksek yetkili bileşenin, düşük yetkili talep sahibi adına yetkisini kullanması
Araç zehirlleme	Araç açıklamasına gizli yönerge yerleştirilerek seçimin yönlendirilmesi
Ayrık model kalıbı	Ayrıcalıklı modelin güvenilmeyen içeriği hiç görmediği savunma
Kısıtlı çözümleme	Çıktının örnekleme aşamasında şemaya zorlanması
İstem önbellegi	İstemin değişmeyen önekinin sağlayıcı tarafında önbellege alınması
İdempotans anahtarı	Yinelenen istekte yan etkinin bir kez uygulanmasını sağlayan anahtar
Devre kesici	Sürekli başarısız bağımlılığı geçici olarak devre dışı bırakan kalıp
Vekâleten yetkilendirme	Etmenin kullanıcı yetkisiyle sınırlı hareket etmesini sağlayan yapı
Jeton değişimi	Geniş kapsamlı jetonun dar kapsamlıyla değiştirilmesi
Kayıt-tekrar	Model 52e araç çağrılarının kaydedilip deterministik yeniden oynatılması
Kanarya dağıtım	Değişikliğin trafiğin küçük bir kısmına verilerek ölçülmesi
Kaynak zinciri	Bir sonucun hangi girdi, sürüm, yetki ve karar zincirinden türediğinin kaydı
Episodik bellek	Olayların zaman sıralı kaydı
Semantik bellek	Olgu ve tercihlerin anahtarlanmış kaydı
Planlı bellek	Yüksek seviyeli planların kaydı

Ek D: Kaynakça Notu

Bu bölüm, metinde anılan kavramların birincil kaynaklarını konu başlıklarına göre gruplandırmaktadır. Metin içi atıflar bu listeye bağlanmalıdır; belgenin kendi kaynak zinciri ilkesi kendi iddiaları için de geçerlidir.

Faillik ve Karar Kuramı

Bandura'nın insan failliği üzerine çalışmaları (sosyal bilişsel kuram, 1986; *Toward a Psychology of Human Agency*, 2006); Simon'ın sınırlı rasyonalite ve satisficing kavramları (*Administrative Behavior*, 1947; *Models of Bounded Rationality*, 1982); Boyd'un OODA döngüsü üzerine sunumları (*A Discourse on Winning and Losing*); March'ın örgütsel öğrenme ve keşif-sömürü dengesi çalışması (1991).

Sibernetik ve Sistem Kuramı

Wiener'in sibernetik üzerine temel eseri (1948); Ashby'nin gerekli çeşitlilik yasası (*An Introduction to Cybernetics*, 1956); Beer'in uygulanabilir sistem modeli (*Brain of the Firm*, 1972); Ostrom'un ortak kaynak yönetişimi çalışması (*Governing the Commons*, 1990).

Klasik Etmen Literatürü

Rao ve Georgeff'in BDI mimarisi üzerine çalışmaları (1991, 1995); Russell ve Norvig'in etmen taksonomisi (*Artificial Intelligence: A Modern Approach*); Smith'in Contract Net Protokolü (1980); Erman ve arkadaşlarının Hearsay-II blackboard mimarisi (1980); Brooks'un subsumption mimarisi ve *Intelligence Without Representation* (1991); Wooldridge ve Jennings'in akıllı etmenler derlemesi (1995); FIPA etmen iletişim dili spesifikasyonları.

Muhakeme Mimarileri

Zincirleme düşünme (Wei ve ark., 2022); ReAct (Yao ve ark., 2022); Reflexion (Shinn ve ark., 2023); Tree of Thoughts (Yao ve ark., 2023); Toolformer (Schick ve ark., 2023).

Hizalama ve Ödül Tasarımı

Amodei ve arkadaşlarının somut güvenlik problemleri çalışması (2016); Krakovna ve arkadaşlarının şartname istismarı örnekleri derlemesi; Soares ve arkadaşlarının düzeltilebilirlik çalışması (2015); Bostrom'un araçsal yakınsama tezi.

Güvenlik

Dolaylı prompt enjeksiyonu üzerine ilk sistematik çalışma (Greshake ve ark., 2023); ayırık model kalıbı üzerine çalışmalar ve yetenek tabanlı türevleri; sınırlayıcı

işaretleme ve spotlighting teknikleri; OWASP’ın büyük dil modeli uygulamaları için risk listesi; Node.js resmî dokümantasyonunun `vm` modülü güvenlik uyarısı ve izin modeli dokümantasyonu; `vm2` projesinin kaçış açıkları nedeniyle sonlandırılma duyurusu; Saltzer ve Schroeder’in en az ayrıcalık ilkesi (1975).

Değerlendirme

SWE-bench (Jimenez ve ark., 2023); WebArena (Zhou ve ark., 2023); GAIA (Mialon ve ark., 2023); model hakem kullanımı ve konum/uzunluk yanlışlıkları üzerine çalışmalar (Zheng ve ark., 2023); kısıtlı çözümleme ve dilbilgisi güdümlü örnekleme üzerine çalışmalar.

İnsan Faktörü

Bainbridge’in otomasyonun ironileri (1983); Parasuraman ve Riley’nin kullanım, kötüye kullanım ve terk çalışması (1997); Endsley’nin durum farkındalığı modeli; Lee ve See’nin otomasyona güven çalışması (2004).

Yazılım Mühendisliği

Olay kaynaklama ve dağıtık sistem kalıpları (Kleppmann, *Designing Data-Intensive Applications*, 2017); Goodhart Yasası’nın ölçüt tasarımına uygulanması.

Düzenleyici Çerçeve

Avrupa Birliği Genel Veri Koruma Tüzüğü’nün otomatik karar verme maddesi; Avrupa Birliği Yapay Zekâ Yasası’nın risk sınıflandırması ve yüksek riskli sistem yükümlülükleri; 6698 sayılı Kişisel Verilerin Korunması Kanunu ve ilgili ikincil düzenlemeler; NIST Yapay Zekâ Risk Yönetimi Çerçevesi.